

Technische Universität Berlin

Multi-Roboter-Systeme

Fakultät IV Elektrotechnik und Informatik



Master Thesis

**Motion Primitive Selection
for Kinodynamic Motion Planning**

Julius Franke

Matriculation Number: 487995

25.11.2024

Supervisors

Akmaral Moldagalieva

Pia Hanfeld

Examiners

Prof. Dr. Wolfgang Hönig

Prof. Dr. Marc Toussaint

Eigenständigkeitserklärung

Hiermit versichere ich, dass ich die vorliegende Arbeit eigenständig ohne Hilfe Dritter und ausschließlich unter Verwendung der aufgeführten Quellen und Hilfsmittel angefertigt habe. Alle Stellen die den benutzten Quellen und Hilfsmitteln unverändert oder sinngemäß entnommen sind, habe ich als solche kenntlich gemacht.

Sofern generative KI-Tools verwendet wurden, habe ich Produktnamen, Hersteller, die jeweils verwendete Softwareversion und die jeweiligen Einsatzzwecke (z.B. sprachliche Überprüfung und Verbesserung der Texte, systematische Recherche) benannt. Ich verantworte die Auswahl, die Übernahme und sämtliche Ergebnisse des von mir verwendeten KI-generierten Outputs vollumfänglich selbst.

Die Satzung zur Sicherung guter wissenschaftlicher Praxis an der TU Berlin vom 8. März 2017¹ habe ich zur Kenntnis genommen.

Ich erkläre weiterhin, dass ich die Arbeit in gleicher oder ähnlicher Form noch keiner anderen Prüfungsbehörde vorgelegt habe.

Berlin, den 25.11.2024


.....
Julius Franke

¹https://www.static.tu.berlin/fileadmin/www/10000060/FSC/Promotion__Habilitation/Dokumente/Grundsaeetze_gute_wissenschaftliche_Praxis_2017.pdf

Self Assertion

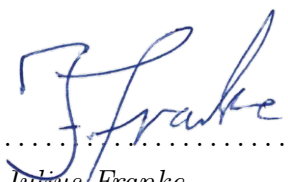
I hereby declare that I have written this thesis independently without the help of third parties and exclusively using the sources and aids listed. I have marked all passages that are taken from the sources and aids used unchanged or in the same sense as such.

If generative AI tools were used, I have indicated the product name, manufacturer, the software version used and the respective purposes (e.g. linguistic review and improvement of the texts, systematic research). I am fully responsible for the selection, adoption and all results of the AI-generated output I used.

I have taken note of the statutes for safeguarding good scientific practice at TU Berlin dated March 8, 2017.²

I further declare that I have not submitted the thesis in the same or a similar form to any other examination authority.

Berlin, 25.11.2024


.....
Julius Franke

²https://www.static.tu.berlin/fileadmin/www/10000060/FSC/Promotion___Habilitation/Dokumente/Grundsatzte_gute_wissenschaftliche_Praxis_2017.pdf

Zusammenfassung

Das Planen sicherer und effizienter Trajektorien unter Berücksichtigung dynamischer Randbedingungen wird als kinodynamische Bahnplanung bezeichnet und ist ein essenzieller Teil der Robotik. Algorithmen wie A*, Randomly Exploring Random Tree (RRT) und k-Order Markov Path Optimization (KOMO) zählen zu den etablierten Lösungsansätzen für derartige Problemstellungen. Neuere Methoden wie Discontinuity Bounded A* (db-A*) und Discontinuity Bounded Conflict-Based Search (db-CBS) kombinieren Merkmale traditioneller Algorithmen und bieten signifikante Verbesserungen in Bezug auf Rechengeschwindigkeit und Ergebnisqualität.

Zentrale Bestandteile dieser Ansätze stellen vordefinierte Trajektoriensegmente dar, sogenannte Motion Primitives, die als Bausteine innerhalb der Bahnplanung genutzt werden. Ihre Auswahl erfolgt in der Regel mittels zufälliger Verfahren, die durch manuell gesetzte Parameter angepasst werden. Dies kann die Effizienz der Planungsalgorithmen jedoch erheblich einschränken, da die resultierenden Trajektorien maßgeblich von der Auswahl der Motion Primitives abhängig sind.

In dieser Arbeit wird ein innovativer Ansatz zur automatisierten Generierung von Motion Primitives, basierend auf Denoising Diffusion Probabilistic Models (DDPMs), vorgestellt. Im Gegensatz zu konventionellen Verfahren lernt dieses Modell, Motion Primitives zu erzeugen, die gezielt auf die spezifischen Anforderungen einer Probleminstanz abgestimmt sind. Da die generierten Datensätze unabhängig vom Planungsalgorithmus erstellt werden, lässt sich dieser Ansatz nahtlos in andere Algorithmen integrieren.

Das vorgestellte Modell kombiniert vier Diffusionsmodelle, um Motion Primitives mit drei verschiedenen Längen zu generieren. Im Vergleich zu Standardmethoden reduzieren die von diesem Modell erzeugten Datensätze die Berechnungszeit um 20% und die Lösungskosten um 25% über eine Vielzahl von Tests. Diese Arbeit zeigt die Vorteile der Nutzung von maschinellem Lernen in der Auswahl von Motion Primitives auf, die wesentlich effizientere und flexiblere Lösungen in der kinodynamischen Bahnplanung erlauben.

Abstract

Determining safe and efficient paths while respecting dynamic constraints is an important part of robotics, known as kinodynamic motion planning. Common methods include algorithms like A*, RRTs, and KOMO. More recent methods like db-A* or db-CBS combine aspects of the more traditional approaches and demonstrate significant improvements in speed and solution quality.

Motion primitives are predefined movements that are often utilized in these algorithms. They serve as components of the solution trajectory and are commonly chosen through random methods or manual adjustments. This traditional process can severely restrict the overall effectiveness of the planning algorithm, as the possible trajectories are limited by the selection of motion primitives.

This thesis presents a novel approach to enhance motion primitive generation using DDPMs. Unlike traditional methods, this model learns to generate motion primitives specifically tailored to a parametrization of each problem instance. This approach can be integrated into other planning algorithms, as the datasets are generated independent of the selected planning algorithm db-A*.

The proposed model combines four diffusion models to allocate and sample motion primitives of three different lengths. When compared to standard methods, the sets of motion primitives generated by this model reduce computation time by 20% and lower solution costs by 25% across various benchmarks. This work highlights the advantages of employing machine learning for motion primitive selection, leading to more efficient and adaptable solution in kinodynamic motion planning.

Contents

List of Figures	xiii
List of Tables	xv
List of Algorithms	xvii
List of Acronyms	xxiii
1 Introduction	1
2 Background	3
2.1 Motion Planning	3
2.1.1 Planning Algorithms	3
2.1.2 Geometric Motion Planning	7
2.1.3 Kinodynamic Motion Planning	8
2.1.4 Motion Primitives	10
2.1.5 Db-A*	12
2.2 Neural Networks	17
2.2.1 Multi Layer Perceptron	17
2.2.2 Diffusion models	18
2.3 Related Work	21
3 Approach	23
3.1 Dataset Creation	23
3.2 Randomly Generating Problem Instances	23
3.3 Training a Basic Model	26
3.3.1 Learning Dynamics	26
3.3.2 Learning Actions Distribution	28
3.4 Extensions	29
3.4.1 Conditioning on Probability	29

3.4.2	Conditioning on Location	30
3.4.3	Conditioning on δ	31
3.4.4	Alternative Representation of θ_0	31
3.4.5	Conditioning on Environment	34
3.4.6	Varying Motion Primitive Lengths	35
4	Results	37
4.1	Setup	37
4.2	Models for Specific Problem Instances	38
4.3	Models for Multiple Problem Instances	43
4.3.1	Parameter Selection	43
4.3.2	Fine-Tuning	45
4.3.3	Composite Model	46
5	Discussion	49
5.1	Models for Specific Problem Instances	49
5.2	Models for Multiple Problem Instances	54
6	Conclusion and Future Work	57
	Bibliography	61
	Appendix A Loss Plots	67

List of Figures

2.1	Example solution of A* in a basic environment	5
2.2	Example solution of RRT in a basic environment	6
2.3	Example optimized RRT solution in a basic environment	7
2.4	Reachable states of a first-order unicycle	10
2.5	Random selection of ten motion primitives for a first-order unicycle	12
2.6	The alcove problem with an example solution	16
2.7	The bugtrap problem with an example solution	16
2.8	The node layout of a basic neural network	17
2.9	Single step of a DDPM with the input and output vector	21
3.1	Two randomly generated problem instances with different obstacle densities	26
3.2	Three example solutions for different discontinuity bounds	31
4.1	Training and validation loss of the dynamics model	39
4.2	Notion of regret for duration and cost for the alcove problem	42
4.3	Notion of regret for duration and cost for the bugtrap problem	42
4.4	Pipeline benchmark	44
4.5	Notion of regret for duration and cost of the top four models	44
4.6	Notion of regret for duration and cost for the fine-tuned model 3	46
4.7	Notion of regret for duration and cost for the composite model	47
5.1	Distribution of s in the alcove and bugtrap training data	51
5.2	Distribution of P_{rel} in the alcove and bugtrap training data	52
A.1	Training and validation loss of the basic model	67
A.2	Training and validation loss of the model with P_{rel}	68
A.3	Training and validation loss of the model with L_{rel}	68
A.4	Training and validation loss of the model with δ	69
A.5	Training and validation loss of the model with Θ_0	69
A.6	Training and validation loss of model 1	70

A.7	Training and validation loss of model 2	70
A.8	Training and validation loss of model 3	70
A.9	Training and validation loss of model 4	71
A.10	Training and validation loss of the tuned model 3	71
A.11	Training and validation loss of the allocator model	71
A.12	Training and validation loss of the composite model length 5	72
A.13	Training and validation loss of the composite model length 10	72
A.14	Training and validation loss of the composite model length 15	72

List of Tables

4.1	Minimum losses for a fixed problem instance	40
4.2	Benchmark results for a fixed problem instance	41
4.3	Hyperparameter selection for the top four models found by Optuna	43
4.4	Minimum losses of the top four models in the parameter selection	44
4.5	Benchmark results for the top four models in the parameter selection . . .	45
4.6	Hyperparameter selection for the fine-tuned model 3	45
4.7	Minimum losses for the fine-tuned model 3 for multiple problem instances	45
4.8	Benchmark results for the fine-tuned model 3 for multiple problem instances	45
4.9	Hyperparameter selection for the composite model	47
4.10	Minimum losses for the composite model for multiple problem instances .	47
4.11	Benchmark results for the composite model for multiple problem instances	48

List of Algorithms

1	A* [2]	5
2	db-A* [5]	14
3	kMP-db-A* [5]	15
4	DDPM Training [9]	19
5	DDPM Sampling [9]	21
6	Random problem instance generation	25

Nomenclature

Sets

$\mathbf{Q}$	Configuration Space
$\mathbf{Q}_{\text{free}}$	Free Configuration Space
$\mathcal{M}$	Motion Primitive
$\mathcal{W}_{\text{blocked}}$	Blocked Workspace
C	Circular obstacle
R	Rectangular obstacle

Matrices

$\mathbf{R}$	Rotation matrix
$\mathbf{W}$	Weights of a layer

Scalars

η	Learning rate
A	Area
b	Bias of a single node
d_{in}	Size of the input layer
d_{out}	Size of the output layer
L_{rel}	Relative location of a motion primitive in a solution
P_{rel}	Relative probability of a motion primitive
r_{c}	Notion of regret for cost [%]
r_{d}	Notion of regret for duration [%]

w	Weight of a neuron connection
z	Activity of a neuron
δ	Discontinuity bound
$\mathcal{W}$	Workspace
$\mathcal{W}_{\text{free}}$	Free Workspace
dt	Time step [s]
θ	Angle of the robot [rad]
φ	Angular velocity [$\frac{\text{rad}}{s}$]
a	Width [m]
e	Height [m]
n	Amount
p	Solution steps
r	Radius [m]
s	Speed [$\frac{m}{s}$]
T	Total duration [s]
t	Time [s]
x	Cartesian x-coordinate [m]
y	Cartesian y-coordinate [m]

Vectors

ϵ	Added noise
ϵ_{θ}	Predicted noise
$\mathbf{a}$	Output of a layer
$\mathbf{b}$	Bias of a layer
$\mathbf{i}$	Input to a layer
$\mathbf{p}_0$	Original data

$\mathbf{p}_{\text{condition}}$ Conditional part of the diffusion input

$\mathbf{p}_{\text{in}}$ Complete input to diffusion model

$\mathbf{p}_t$ Noised data

$\mathbf{v}_{\text{rot}}$ $\mathbf{e}_1$ rotated by θ_0

$\mathbf{w}$ Weights of a single node

$\mathbf{z}$ Activity of a layer

Θ_0 Angle of the robot as first column of rotation matrix

$\mathbf{q}$ Configuration

$\mathbf{q}_{\text{goal}}$ Goal configuration

$\mathbf{q}_{\text{start}}$ Start configuration

Other

$\mathbf{g}$ Inequality constraint

$\mathbf{h}$ Equality constraint

f_d Discrete dynamics function

g Activation function

J Objective/Cost Function

$\hat{h}$ Perfect heuristic

$\mathcal{B}$ Configuration Map

f Dynamics Function

h Heuristic function

List of Acronyms

ADAM	Adaptive Moment Estimation
ASHA	Asynchronous Successive Halving
CDF	Cumulative Distribution Function
CHOMP	Covariant Hamiltonian Optimization for Motion Planning
db-A*	Discontinuity Bounded A*
db-CBS	Discontinuity Bounded Conflict-Based Search
DDPM	Denoising Diffusion Probabilistic Model
DMP	Dynamic Movement Primitive
IQR	Interquartile Range
KOMO	k-Order Markov Path Optimization
MLP	Multi Layer Perceptron
MSE	Mean Squared Error
PRM	Probabilistic Roadmap
ProDMP	Probabilistic Dynamic Movement Primitive
ProMP	Probabilistic Movement Primitive
ReLU	Rectified Linear Unit
RL	Reinforcement Learning
RRT	Randomly Exploring Random Tree

1 Introduction

Kinodynamic motion planning [1] is a crucial part of robotics. It aims to find motion strategies that guide robots from a start to a specified goal while adhering to the robot’s dynamic constraints. It simultaneously optimizes an objective, such as time or energy consumption. Traditional methods to address this motion planning problem include graph-based methods such as A* [2], sampling-based approaches like Randomly Exploring Random Tree (RRT) [3], or optimization-based methods like k-Order Markov Path Optimization (KOMO) [4]. While these approaches are effective in various scenarios, hybrid approaches like Discontinuity Bounded A* (db-A*) [5] and Discontinuity Bounded Conflict-Based Search (db-CBS) [6] have shown significant improvements in both computational efficiency and solution quality.

A vital component of these hybrid approaches is the use of so-called motion primitives, which are short, pre-computed motions that obey the dynamic constraints of the robot. The planning algorithm then uses a set of such motion primitives to construct a valid trajectory, with the motion primitives as building blocks. The selection of this set is a significant decision, which has shown to have an empirical impact on the computation time of the planning algorithm and the solution cost [7][8]. Current methods for this selection are mostly based on random selection, including tuning of some hyperparameters for this selection process, which may not fully utilize the planner’s potential.

This thesis aims to optimize this motion primitive selection and extend the planner’s potential by leveraging Denoising Diffusion Probabilistic Models (DDPMs) [9], a type of generative machine learning model. Conditioned on the problem instance, the model’s objective is to automatically generate a set of motion primitives whose trajectories are tailored to the problem’s specifications, unlike traditional approaches. Implementing the diffusion model decoupled from the specific planning algorithm db-A* ensures that this generative approach to motion primitive generation can seamlessly be incorporated into other existing planners.

The evaluation of the proposed model and its generated set of motion primitives is realized by the existing planning algorithm db-A* across many benchmarks. Comparing

the performance of traditionally generated motion primitives with the diffusion model method, this thesis will demonstrate the perks of applying learning-based methods to generate motion primitives for kinodynamic motion planning. The model’s proposed ability to generalize to arbitrary problem instances further displays these benefits.

This work seeks to bridge the divide between heuristic and learning-based approaches to motion primitive selection, enabling the development of more robust and efficient solutions for kinodynamic motion planning. The evaluation results of this approach show that DDPMs are capable of learning the distribution of solution trajectories comprised of translation-invariant motion primitives. By conditioning on various problem-describing parameters, the motion primitive datasets generated by this approach allow db-A* to find a solution 20% faster and a 25% lower cost than the baseline.

2 Background

The approach presented in this thesis combines two research areas – Motion Planning and Machine Learning. In this chapter, relevant topics from both are introduced.

2.1 Motion Planning

Motion planning in robotics is the process of translating high-level tasks into low-level instructions that can be executed by a robot [10]. Common tasks are reaching a given location or manipulating an object in a specific way. The objective is to find a collision-free path that solves the given task, trying to find the most efficient solution.

The robot’s current status is described by its state, or configuration $\mathbf{q} \in \mathcal{Q} \subset \mathbb{R}^d$, with d being the dimensionality of the configuration $\mathbf{q}$. The set $\mathcal{Q}$ is called the configuration space and contains all possible configurations. Another alternative notation is $\mathbf{x} \in \mathcal{X} \subset \mathbb{R}^d$, which this thesis will not use [11]. The robot’s configuration is optimally the minimal vector to represent its current state. This state can include, for example, the robot’s position in space or its speed. The subset $\mathcal{Q}_{\text{free}} \subset \mathcal{Q}$ contains all states in which the robot does not collide with the environment.

2.1.1 Planning Algorithms

There are many planning algorithms available to solve motion planning problems. Three approaches are generally used - graph-based, sampling-based, and optimization-based. These approaches are not exclusive and can be combined.

Graph-Based Planners

Graph-based approaches use a graph representation of the configuration space to find a solution. Due to its mathematical properties, the graph pathfinding algorithm A^* [2] is a popular choice. A^* is *complete*, meaning it will find a solution if one exists, and it is also *optimal*, meaning it will find the most optimal solution [12]. The optimal solution is

given by a predefined objective function J . This function is often a measurement of either the solution length, time, or energy. A^* requires a heuristic function h to evaluate new states. Completeness and optimality are important properties of planning algorithms. All graph-based algorithms have the major drawback of requiring a representation of the configuration space as a graph. Constructing such a graph is not a trivial problem and needs to be computed by other algorithms.

A^* is used to find the shortest path between two nodes in a weighted graph. The pseudo-code in Alg. 1 outlines the algorithm. The algorithm requires the nodes $\mathbf{q}_{\text{start}}$ and $\mathbf{q}_{\text{goal}}$, representing the start and goal. Two sets of graph nodes will be stored in memory, the open set $\mathcal{O}$ and the closed set $\mathcal{C}$. At first, the start node $\mathbf{q}_{\text{start}}$ is used to initialize the open set. There are two ways A^* will exit: if the currently selected node n is the goal node $\mathbf{q}_{\text{goal}}$, or if $\mathcal{O} = \emptyset$. The latter is the result of an infeasible problem. The former signals the found solution. Each step picks the node with the lowest f-value from the open set. The f-value is the sum of the observed actual cost-to-come g and the estimated cost to the goal h . The cost-to-go is estimated based on a heuristic function, such as the Euclidean distance to the goal. The heuristic function h plays an important part in the algorithm's performance and properties. The optimality of A^* depends on h , specifically on the condition that h is admissible. Given the unknown, actual cost-to-go $\hat{h}$, h is admissible if $\hat{h} \leq h$ [13]. Given an admissible heuristic h , A^* exits with the optimal solution. Combining known weights with unknown, estimated weights, A^* , can balance exploration with exploitation.

In Fig. 2.1, an example for A^* is shown. The black outline defines the workspace $\mathcal{W}$, and the black blocks in the center are the obstacles. As mentioned earlier, generating the graph and the placement of its nodes is not part of A^* . The graph and its components were placed manually in this example. The task is to find the shortest path from $\mathbf{q}_{\text{start}}$ to $\mathbf{q}_{\text{goal}}$. The distance between the vertices is shown on the edges. In this example, the configuration $\mathbf{q}_5$ is not explored by A^* , as it found the solution before expanding into that node. The solution is marked in red, while the explored nodes are marked in yellow.

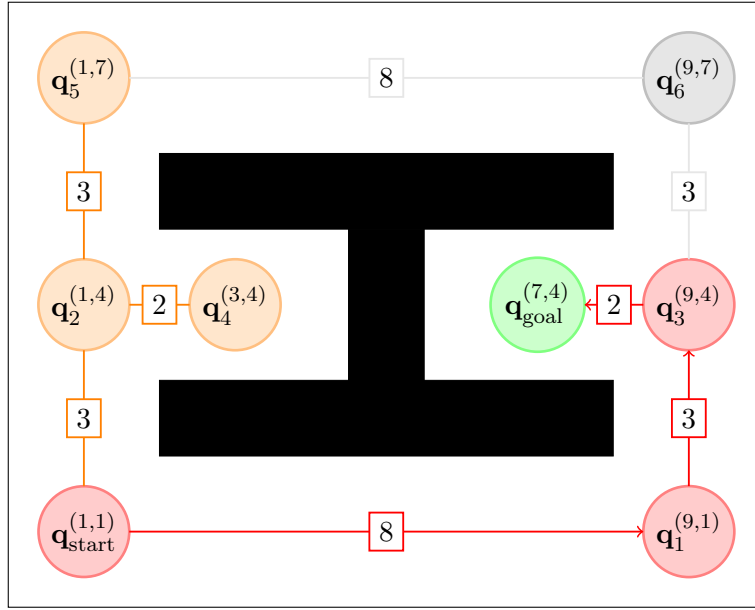


Figure 2.1: An example solution of A* in a basic environment (start $\mathbf{q}_{\text{start}}$, goal $\mathbf{q}_{\text{goal}}$, solution *red*, explored *orange*)

Algorithm 1 A* [2]

Input: $\mathbf{q}_{\text{start}}, \mathbf{q}_{\text{goal}}$

Output: $\mathbf{Q}_d, \mathbf{U}_d$ or Infeasible

```

1:  $\mathcal{O} \leftarrow \{\text{Node}(\mathbf{q} : \mathbf{q}_{\text{start}}, g : 0, h : h(\mathbf{q}_{\text{start}}), p : \text{None})\}$   $\triangleright$  Initialize open priority queue
2:  $\mathcal{C} \leftarrow \emptyset$   $\triangleright$  Initialize closed set
3: while  $|\mathcal{O}| > 0$  do
4:    $n \leftarrow \text{PRIORITYQUEUEPOP}(\mathcal{O})$   $\triangleright$  Lowest  $f$ -value
5:   if  $n.\mathbf{q} = \mathbf{q}_{\text{goal}}$  then
6:     return  $\mathbf{Q}_d, \mathbf{U}_d, T_d$   $\triangleright$  Lowest  $f$ -value
7:    $\text{CLOSEDSETADD}(\mathcal{C}, n)$ 
8:   for  $n' \in \text{SUCCESSOR}(n)$  do
9:     if  $n' \in \mathcal{C}$  then
10:      continue
11:      $g_t \leftarrow n.g + \text{cost}(n, n')$   $\triangleright$  tentative  $g$  score for this action
12:     if  $s < n'.g$  then
13:        $n'.g = g_t$   $\triangleright$  Update cost
14:        $n'.p = n$   $\triangleright$  Update parent
15:       if  $n' \in \mathcal{O}$  then
16:          $\text{PRIORITYQUEUEUPDATE}(\mathcal{O}, n')$ 
17:       else
18:          $\text{PRIORITYQUEUEINSERT}(\mathcal{O}, \text{Node}(\mathbf{q} : n'.\mathbf{q}, g : g_t, h : h(n'), p : n))$ 
19: return Infeasible

```

Sampling-Based Planners

Sampling-based planners are based on the idea of exploring the free configuration space $\mathcal{Q}_{\text{free}}$ by sampling configurations. Probabilistic Roadmap (PRM) [14] and Randomly Exploring Random Trees (RRTs) [3] are two examples for sampling-based planning algorithms. An example solution given by RRT is shown in Fig. 2.2. The final solution is marked in red. The nodes marked in orange were sampled but ultimately not used. A sampling-based approach cannot achieve full completeness and optimality [15], as they are not deterministic. They can, however, achieve some limited form of these properties. RRT* [16] is an extension of RRT. It can be shown that RRT* is *probabilistic* complete. If one exists, it will find a solution for the number of samples towards infinity. Under the same condition, RRT* is *asymptotically* optimal [17]. It will converge to the optimal solution with increasing samples.

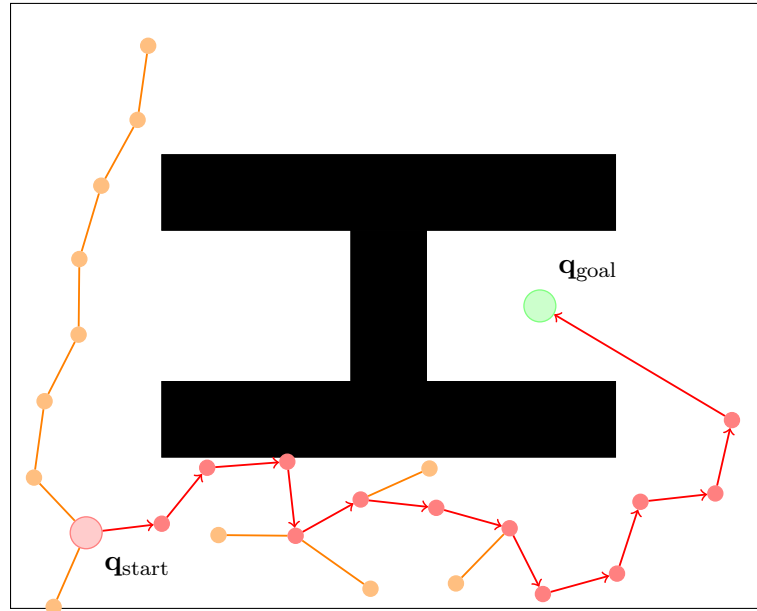


Figure 2.2: An example solution of RRT in a basic environment (start $\mathbf{q}_s$, goal $\mathbf{q}_g$, solution *red*, explored *orange*)

Optimization-Based Planners

Planning algorithms that formulate the problem as an optimization problem are called optimization-based. Both gradient-based and gradient-free approaches are used. The gradient is the first derivative of a predefined cost function J , which is used to approach

the optimal solution in steps that require an initial guess. Calculating the cost function's gradient is often costly or sometimes not even feasible. Gradient-free approaches circumvent this by only using the cost function to explore the configuration space. Two examples of gradient-based approaches are KOMO [4] and Covariant Hamiltonian Optimization for Motion Planning (CHOMP) [18]. An example of a gradient-free approach is shown in [19]. In contrast to both sampling-based and graph-based planners, the configuration space is not discretized, which often leads to smoother, higher-quality solutions. An example of how the solution given by RRT in Fig. 2.2 would be changed by optimization is shown in Fig. 2.3. This example is a high-level illustration not based on an actual solution from an optimization-based planner. The original solution's nodes (orange) are slowly moved along the gradient (black), of the cost function ∇J , leading to the optimized solution (red). However, the optimization problems formulated by the motion planning problem, are often non-convex and highly dependent on the initial guess [20][21].

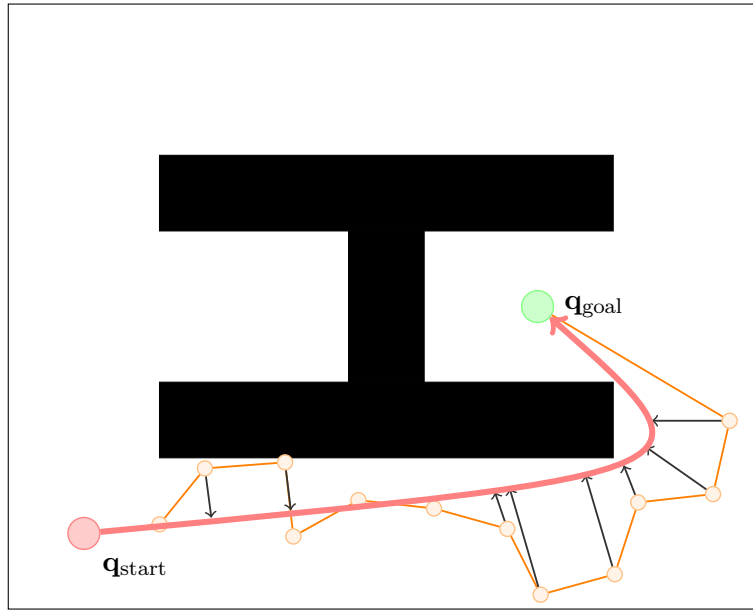


Figure 2.3: An example optimized RRT solution in a basic environment (start $\mathbf{q}_{\text{start}}$, goal $\mathbf{q}_{\text{goal}}$, starting solution *orange*, final solution *red*)

2.1.2 Geometric Motion Planning

Motion planning can be divided into geometric and kinodynamic motion planning. Geometric motion planning describes the process of finding a geometric route for the given

task, only considering the robot’s collision shape and environment. Given an objective function J , states $\mathbf{q}_{\text{start}}$ and $\mathbf{q}_{\text{goal}}$ and the free configuration-space $\mathcal{Q}_{\text{free}}$, the goal defined in [22] is to

$$\begin{aligned} \min J(\mathbf{q}(p)) \quad & \text{s.t.} \\ \mathbf{q}(0) &= \mathbf{q}_{\text{start}} \\ \mathbf{q}(1) &= \mathbf{q}_{\text{goal}} \\ \mathbf{q}(p) &\subset \mathcal{Q}_{\text{free}} \quad \forall p \in [0, 1]. \end{aligned} \tag{2.1}$$

In most cases, p is not continuous but discrete. Calculating only a finite amount of steps instead of a continuous function greatly decreases the complexity of the problem. No dynamics are considered, and thus p is independent of time. The time of the solution states is usually determined after the solution path has already been found. Depending on the problem, there may be no direct way to know $\mathcal{Q}_{\text{free}}$. The free workspace is usually unknown if the robot’s state space is not the same as the workspace, for example, if a rotation is needed to describe the robot’s configuration. A configuration map $\mathcal{B}(\mathbf{q})$ maps the robot’s state $\mathbf{q}$ onto the workspace $\mathcal{W}$. Given $\mathcal{B}(\mathbf{q})$ and the free workspace $\mathcal{W}_{\text{free}}$, the last constraint of Eq. 2.1 can be rewritten as:

$$\mathcal{B}(\mathbf{q}(p)) \subset \mathcal{W}_{\text{free}} \quad \forall p \in [0, 1]. \tag{2.2}$$

For the geometric case, the total distance of the found path is usually a good pick for the objective function. Depending on the composition of the robot’s state, this can, for example, be the basic Euclidean distance or a rotation distance.

2.1.3 Kinodynamic Motion Planning

In the geometric case, the result of the motion planning problem only contains the waypoints to the solution without information on how to reach them. This information is sufficient for some types of robots but insufficient for others. A vacuum robot can turn on the spot and drive a straight line to any desired configuration. The actions $\mathbf{u}$ required to reach the next state can be deterministically computed. A car, however, has to steer and turn to get to any configuration apart from moving straight ahead or behind. A configuration directly to the side of a car has at least two equivalent solutions – going forward or backward. The dynamics of the car are too complex to ignore them, therefore the problem definition has to be expanded.

In contrast to geometric motion planning, kinodynamic motion planning aims to find a solution while respecting the dynamic constraints of the robot and its environment. The dynamics of a robot describe how the robot behaves under specific circumstances, subject to time. Generally, dynamics are represented by a function that describes the rate of change of the state $\dot{\mathbf{q}}(t)$, given the current state $\mathbf{q}(t)$ and the control input $\mathbf{u}(t)$ at a specific time step t :

$$\dot{\mathbf{q}} = f(\mathbf{q}, \mathbf{u}). \quad (2.3)$$

First-Order Unicycle

A simple example of robot dynamics is the first-order unicycle, proposed in [23], in a 2-dimensional workspace. The minimal vector for the state of the unicycle is $\mathbf{q} = (x, y, \theta)$, with the control input $\mathbf{u} = (s, \varphi)$. The dynamics equation is as follows:

$$\dot{\mathbf{q}} = \begin{pmatrix} s \cos(\theta) \\ s \sin(\theta) \\ \varphi \end{pmatrix} \quad (2.4)$$

It is called a **first-order** unicycle because the control input consists of the first-order derivatives of x, y and θ , its velocity s , and its angular velocity φ . A second-order unicycle control input would control the acceleration. Usually, the control inputs are not unlimited. Fig. 2.4 shows all possible states a unicycle can reach within 5 seconds for $\mathbf{q}_{\text{start}} = [0 \ 0 \ 0]^\top$, $s \in [-0.5, 0.5]$ and $\varphi \in [-0.5, 0.5]$, given time-invariant control inputs.

Problem Formulation

The goal of a kinodynamic motion planning problem, according to [22], is to find

$$\begin{aligned} \min J(\mathbf{u}(t), \mathbf{q}(t)) \quad & \text{s.t.} \\ \mathbf{q}(0) = \mathbf{q}_{\text{start}} \quad & \mathbf{q}(T) = \mathbf{q}_{\text{goal}} \\ \mathcal{B}(\mathbf{q}(t)) \subset \mathcal{W}_{\text{free}} \quad & \forall t \in [0, T] \\ \dot{\mathbf{q}}(t) = \mathbf{f}(\mathbf{q}(t), \mathbf{u}(t)) \quad & \forall t \in [0, T]. \end{aligned} \quad (2.5)$$

With the inclusion of the robot dynamics, the problem is now time-dependent, as opposed to the geometric case of $p \in [0, 1]$. This information can be used to evaluate the found solution based on the time it takes the robot to complete the task. Being

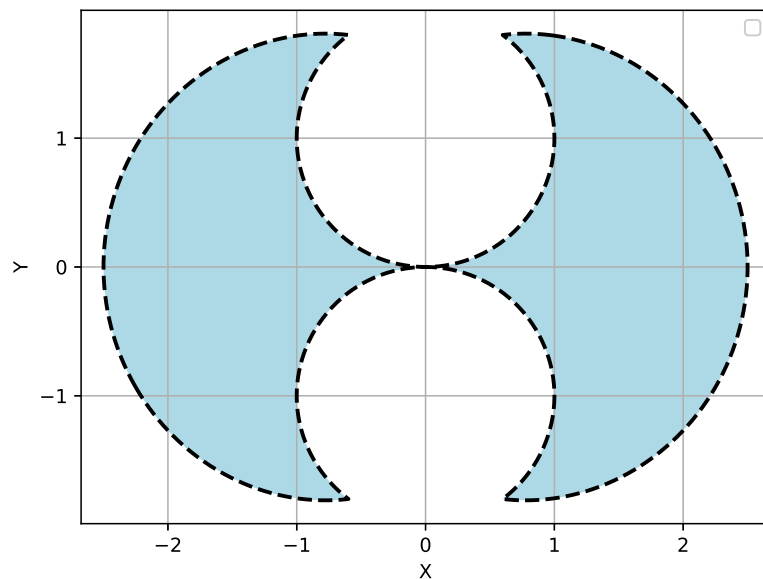


Figure 2.4: Possible states a first-order unicycle can reach within 5 seconds given time-invariant control inputs

dependent on a differential equation introduces a new complexity that must be addressed. The differential equation must be solved to compute the control input $\mathbf{u}(t)$ from one state to another. In some cases, this can be analytically solved by the process called inverse kinematics [24]. Depending on the robot dynamics, there also may be multiple equivalent solutions to reach a specific state. In most cases, solving this problem for every step of the motion planning algorithm is impractical and solved in some other way. One solution to this is the usage of motion primitives.

2.1.4 Motion Primitives

A motion primitive is a generally basic, predefined movement consisting of a set of actions $\mathbf{u}(t)$ and states $\mathbf{q}(t)$ [25]. The states are linked to the actions by the dynamics f . They are usually defined as discrete values, calculated, for example, by Euler steps:

$$\begin{aligned}
 \mathbf{q}_{i+1} &= f_d(\mathbf{q}_i, \mathbf{u}_i) \quad \forall i \in [0, n] \\
 &= \mathbf{q}_i + \dot{\mathbf{q}}_i dt \quad (f_d \text{ Euler step}) \\
 T &= ndt,
 \end{aligned} \tag{2.6}$$

with T being the total duration, n the amount of discrete steps and dt the time-step size of that specific primitive. A single motion primitive is defined as a tuple of n actions $\mathbf{u}_i$ and $n + 1$ states $\mathbf{q}_i$:

$$\mathcal{M} = \left\{ [\mathbf{q}_0, \mathbf{q}_1, \dots, \mathbf{q}_n], [\mathbf{u}_0, \mathbf{u}_1, \dots, \mathbf{u}_{n-1}] \right\}. \quad (2.7)$$

They are usually computed before the actual planning process. In most cases, a large dataset of motion primitives is randomly generated for a specific robot. In the planning process, they are used as building blocks for the solution. Optimally, each motion primitive of the dataset should represent a basic maneuver so the robot can reach any possible location.

The key idea behind motion primitives is to simplify the motion planning process. Rather than calculating every little detail of a robot's path on the fly, the planner chooses from a set of predefined, optimized movements. For instance, the first-order unicycle might have primitives for a right turn, a left turn, and driving straight ahead. When computing a solution, the motion planning algorithm can select from these pre-calculated maneuvers instead of calculating every single steering angle or speed change [26][27]. Thus, the problem can be boiled down to a graph search with a known number of vertices [28]. This approach is helpful for robots with complex dynamics or dynamic environments where quick decisions are needed since selecting and combining motion primitives is typically faster than computing an entirely new path from scratch.

However, the solution quality and success rate are highly dependent on the available motion primitives. If a particular maneuver is needed to, for example, pass through a narrow passage, but no such primitive is in the dataset, no solution will be found. The motion primitives shown in Fig. 2.5 were generated by randomly sampling control inputs. In this example, there are no primitives that let the unicycle drive straight or make a right turn while moving backward. By simply generating more primitives, a scenario where a specific motion is missing might be avoided. Depending on the dynamics complexity and the configuration space's dimensionality, the number of motion primitives needed to reach every possible state can explode quickly. Making sure to sample uniformly over the action space can help, but most often, this will not lead to a uniform distribution in the configuration space.

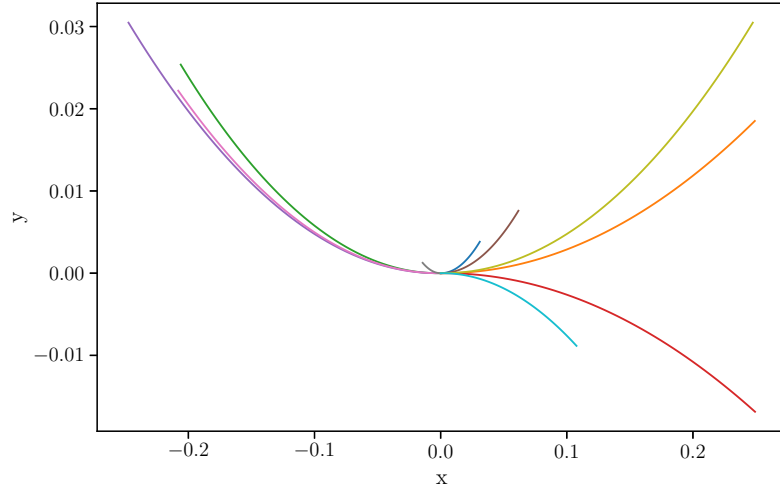


Figure 2.5: Random selection of ten motion primitives for a first-order unicycle allows no straight/backward or backward/right movement

2.1.5 Db-A*

The motion planning algorithm db-A* is an extension of the well-known A* algorithm [5]. It combines A* with optimization-based methods, constructing a graph at runtime using motion primitives as edges. The algorithm relies on the fact that many robot dynamics are translation invariant, which allows for reusing motion primitives by shifting them. Using precomputed motion primitives as edges can lead to many nodes having no applicable motion primitives. With random (or semi-random) generation of the dataset, the probability that the end-state of a motion primitive has no matching start-state of another can be quite high. This is especially noticeable with more complex dynamics. Traditionally, a vertex on the graph is a specific point in the configuration space. Db-A*, however, allows for bounded discontinuities at the vertices. This can be seen in Alg. 2 (differences to A* marked in yellow) on line 6, with $d(n.\mathbf{q}, \mathbf{q}_f) \leq \delta$. The allowed discontinuity is the user-specified value δ . A second user-specified parameter α sets a trade-off between pruning a new state in favor of an existing one and keeping it (Line 15). The mathematical properties of A*, as described in Sec. 2.1.1 do not hold for $\delta > 0$, as the random order of the motion primitives does not guarantee to find a solution, even if there is one. In this case, db-A* is incomplete and, as such, also not optimal. However, as $\delta \rightarrow 0$, the properties of A* apply again.

Solution paths with discontinuities are not applicable on a real robot. Therefore, the

solutions given by db-A* have to be modified to make them continuous. This optimization is done with KOMO [4], which repairs the discontinuities. Given the sequence of state $\mathbf{q}_{l-k:l} = \mathbf{q}_{l-k}, \mathbf{q}_{l-k+1}, \dots, \mathbf{q}_l$, the inequality constraints $\mathbf{g}_l$ and the equality constraints $\mathbf{h}_l$, the repaired path is computed by solving

$$\begin{aligned} \min_{\mathbf{q}} \sum_{l=1}^T \hat{J}(\mathbf{q}_{l-k:l}) \quad \text{s.t.} \\ \mathbf{g}_l(\mathbf{q}_{l-k:l}) \leq \mathbf{0} \quad \forall l \in \{1, \dots, T\} \\ \mathbf{h}_l(\mathbf{q}_{l-k:l}) = \mathbf{0} \quad \forall l \in \{1, \dots, T\}. \end{aligned} \tag{2.8}$$

Combining these two approaches allows for fast initial solutions with a large δ and better solutions with $\delta \rightarrow 0$. Utilizing this, Alg. 3 continuously generates solutions with increasingly lower cost. Motion primitives generated through optimization are included in $\mathcal{M}$ for the next run with a smaller δ . This results in an *anytime* motion planning algorithm. *Anytime* refers to the fact that a planning algorithm can quickly find an initial solution on which it can improve by investing more time [29]. A solution can be extracted at *any time*, usually with a trade-off between computing time and cost.

Algorithm 2 db-A* [5]

Input: $\mathbf{q}_{\text{start}}, \mathbf{q}_{\text{goal}}, \mathcal{Q}_{\text{free}}, \mathcal{M}, \delta, c_{\text{max}}$
Output: $\mathbf{Q}_d, \mathbf{U}_d$ or Infeasible

```

1:  $\mathcal{T}_m \leftarrow \text{NEARESTNEIGHBORINIT}(\mathcal{M})$   $\triangleright$  Use start states of motions excl. position
2:  $\mathcal{T}_n \leftarrow \text{NEARESTNEIGHBORINIT}(\{\mathbf{q}_{\text{start}}\})$   $\triangleright$  capture explored vertices incl. position
3:  $\mathcal{O} \leftarrow \{\text{Node}(\mathbf{q} : \mathbf{q}_{\text{start}}, g : 0, h : h(\mathbf{q}_{\text{start}}), p : \text{None}, a : \text{None})\}$   $\triangleright$  Initialize open priority queue
4: while  $|\mathcal{O}| > 0$  do
5:    $n \leftarrow \text{PRIORITYQUEUEPOP}(\mathcal{O})$   $\triangleright$  Lowest f-value
6:   if  $d(n.\mathbf{q}, \mathbf{q}_{\text{goal}}) \leq \delta$  then
7:     return  $\mathbf{Q}_d, \mathbf{U}_d, T_d$   $\triangleright$  Lowest f-value
8:    $\triangleright$  Find applicable motion primitives with discontinuity up to  $\alpha\delta$   $\triangleleft$ 
9:    $\mathcal{M}' \leftarrow \text{NEARESTNEIGHBORQUERY}(\mathcal{T}_m, n \cdot \mathbf{q}^\top, \alpha\delta)$ 
10:  for  $m \in \mathcal{M}'$  do
11:    if  $n.\mathbf{q} \oplus m \notin \mathcal{Q}_{\text{free}}$  then
12:      continue  $\triangleright$  entire motion is not collision-free
13:       $g_t \leftarrow n.g + \text{cost}(m)$   $\triangleright$  tentative g score for this action
14:       $\triangleright$  find already explored nodes within  $(1 - \alpha)\delta$   $\triangleleft$ 
15:       $\mathcal{N}' \leftarrow \text{NEARESTNEIGHBORQUERY}(\mathcal{T}_n, n \cdot \mathbf{x} \oplus m, (1 - \alpha)\delta)$ 
16:      if  $\mathcal{N}' = \emptyset$  then
17:         $\text{PRIORITYQUEUEINSERT}(\mathcal{O}, \text{Node}(\mathbf{q} : n \cdot \mathbf{q} \oplus m, g : g_t, h : h(n \cdot \mathbf{q} \oplus m), p : n, a : m))$ 
18:         $\text{NEARESTNEIGHBORADD}(\mathcal{T}_n, n.\mathbf{q} \oplus m)$ 
19:      else
20:        for  $n' \in \mathcal{N}'$  do
21:          if  $g_t < n'.g$  then  $\triangleright$  This motion is better than a known motion
22:             $n'.g = g_t$   $\triangleright$  Update cost
23:             $n'.p = n$   $\triangleright$  Update parent
24:             $n'.a = m$   $\triangleright$  Update action
25:           $\text{PRIORITYQUEUEUPDATE}(\mathcal{O}, n')$ 
26: return Infeasible

```

Algorithm 3 kMP-db-A* [5]

```

1:  $\mathcal{M} \leftarrow \emptyset$   $\triangleright$  Set of motion primitives
2:  $c_{\max} = 1, 2, \dots \leftarrow \infty$   $\triangleright$  Solution cost bound
3: for  $n = 1, 2, \dots$  do
4:    $\mathcal{M} \leftarrow \mathcal{M} \cup \text{ADDP}(\text{PRIMITIVES})$ 
5:    $\delta \leftarrow \text{COMPUTED}(\Delta)(\mathcal{M})$ 
6:    $\mathbf{Q}_d, \mathbf{U}_d, T_d \leftarrow \text{DB-A}^*(\mathbf{q}_{\text{start}}, \mathbf{q}_{\text{goal}}, \mathcal{Q}_{\text{free}}, \delta, c_{\max})$ 
7:   if  $\mathbf{Q}_d, \mathbf{U}_d$  successfully computed then
8:      $\mathbf{Q}, \mathbf{U}, T \leftarrow \text{OPTIMIZATION}(\mathbf{Q}_d, \mathbf{U}_d, T_d, c_{\max})$ 
9:     if  $\mathbf{X}, \mathbf{U}$  successfully computed then
10:       $\text{REPORT}(\mathbf{Q}, \mathbf{U}, T)$   $\triangleright$  New solution found
11:       $c_{\max} \leftarrow \min(c_{\max}, J(\mathbf{Q}, \mathbf{U}, T))$   $\triangleright$  cost bound
12:       $\mathcal{M} \leftarrow \mathcal{M} \cup \text{EXTRACT}(\text{PRIMITIVES})(\mathbf{Q}, \mathbf{U})$ 

```

A problem instance is defined by the combination of an *environment* and a task description, usually given by a starting state $\mathbf{q}_{\text{start}}$ and a goal state $\mathbf{q}_{\text{goal}}$. The implementation of db-CBS [6] defines a problem in the single robot case by:

$$\begin{aligned}
& \min J(\mathbf{u}(t), \mathbf{q}(t), T) \quad \text{s.t.} \\
& \mathbf{q}(0) = \mathbf{q}_{\text{start}} \quad \mathbf{q}(T) = \mathbf{q}_{\text{goal}} \\
& \mathcal{W} = \{(x, y) \in \mathbb{R}^2 \mid x_{\text{lb}} \leq x \leq x_{\text{ub}} \text{ and } y_{\text{lb}} \leq y \leq y_{\text{ub}}\} \\
& \mathcal{W}_{\text{blocked}} = \left\{ (x, y) \in \mathbb{R}^2 \mid (x, y) \in \bigcup_{i=1}^{n_R} R_i \cup \bigcup_{j=1}^{n_C} C_j \right\} \\
& R_i = \{(x, y) \in \mathbb{R}^2 \mid \\
& \quad x_i - \frac{a_i}{2} \leq x \leq x_i + \frac{a_i}{2} \text{ and } y_i - \frac{e_i}{2} \leq y \leq y_i + \frac{e_i}{2}\} \\
& C_i = \left\{ (x, y) \in \mathbb{R}^2 \mid \sqrt{(x - x_j)^2 + (y - y_j)^2} \leq r_j \right\} \\
& \mathcal{W}_{\text{free}} = \mathcal{W} \setminus \mathcal{W}_{\text{blocked}} \\
& \mathcal{B}(\mathbf{q}(t)) \subset \mathcal{W}_{\text{free}} \quad \forall t \in [0, T] \\
& \mathbf{q}_{i+1} = f_d(\mathbf{q}_i, \mathbf{u}_i).
\end{aligned} \tag{2.9}$$

The workspace $\mathcal{W}$, set in $\mathbb{R}^2$, is defined as a rectangle given by its corner coordinates $(x_{\text{lb}}, y_{\text{lb}})$ and $(x_{\text{ub}}, y_{\text{ub}})$. The free workspace is given by $\mathcal{W} \setminus \mathcal{W}_{\text{blocked}}$ with the blocked workspace consisting of a set of rectangles R_i and circles C_j . A single rectangle R_i is described by its center (x_i, y_i) and its extents w_i and h_i . Similarly, a circle C_j is given by

its center (x_j, y_j) and its radius r_j . The problem description is given by $\mathbf{q}_{\text{start}}$ and $\mathbf{q}_{\text{goal}}$. The type of robot defines the dynamics function f_d . The objective is to minimize the cost function J . While db-CBS allows for the multi-robot case, db-A*, and by extent, this thesis only considers the single robot case.

Two examples of problem instances defined by Eq. 2.9 are shown below. The alcove problem is presented in Fig. 2.6 and the bugtrap problem in Fig. 2.7. They additionally include a solution given by db-A* for the first-order unicycle presented in Sec. 2.1.3. The start and goal configurations $\mathbf{q}_{\text{start}}$ and $\mathbf{q}_{\text{goal}}$ are visualized as arrows, indicating their respective angle. The solution is shown as a line, where alternating colors of red and teal mark the separate motion primitives. The discontinuity bound set to $\delta = 0.5$ for both examples is shown as a dotted orange line.

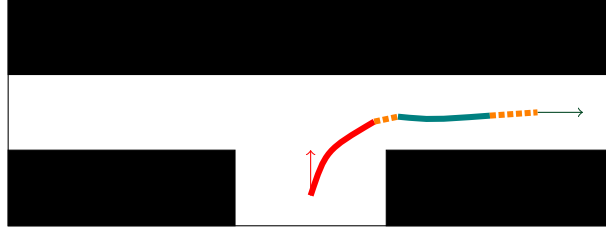


Figure 2.6: The alcove problem with an example solution highlighting the separate motion primitives (start *red* arrow, goal *green* arrow, solution *red* and *teal*, discontinuity *orange*)

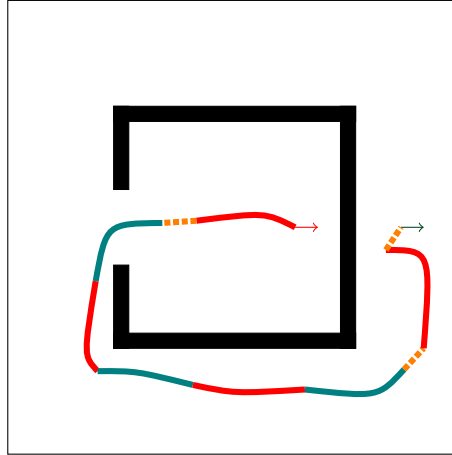


Figure 2.7: The bugtrap problem with an example solution highlighting the separate motion primitives (start *red* arrow, goal *green* arrow, solution *red* and *teal*, discontinuity *orange*)

2.2 Neural Networks

Neural networks are a commonly used method in machine learning, as they can learn complex structures and patterns from data. They are used in a wide variety of fields, most notably as natural language processors and in computer vision. They can learn these patterns in data because they are universal function approximators [30][31]. However, this is limited to the model complexity and the training data available.

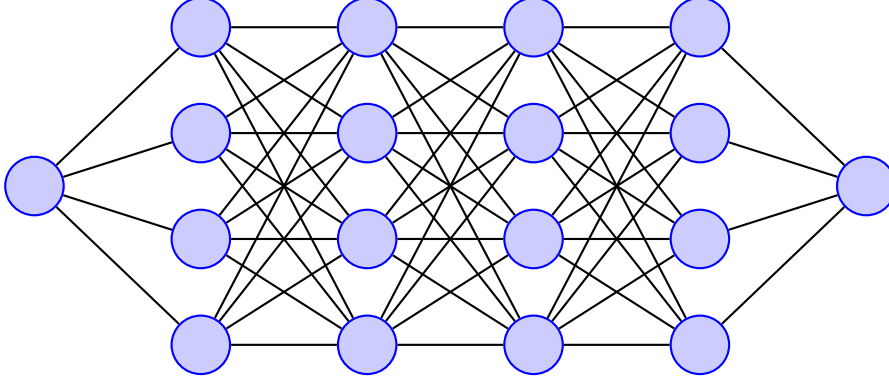


Figure 2.8: The node layout of a basic neural network

2.2.1 Multi Layer Perceptron

The basic structure of a machine learning model, a Multi Layer Perceptron (MLP) [32], can be seen in Fig. 2.8. It is comprised of nodes organized in the so-called layers. The leftmost layer is called the input layer, and the rightmost layer is the output layer. The layers in between input and output are referred to as hidden layers. All nodes j of one layer l are connected by an edge with the weight w_{ij} . Each node is parametrized by an activation function g and a bias b . The output of a single node is given by $g(z)$ with

$$\begin{aligned} z &= \sum_{j=1}^n w_j i_j + b \\ &= w_1 i_1 + w_2 i_2 + \dots + w_n i_n + b \end{aligned} \tag{2.10}$$

The activation function g is chosen depending on the problem at hand. Some examples are the sigmoid function $\sigma(z) = \frac{1}{1+e^{-z}}$ and the Rectified Linear Unit (ReLU) $\text{ReLU}(z) = \max(0, z)$. To further compact the computation of a layer, the output of an entire layer l can be written as

$$g(\mathbf{z}^{(l)}) = g(\mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}) \tag{2.11}$$

with

$$\mathbf{a}^l = g\left(\mathbf{z}^{(l)}\right) \quad (2.12)$$

The output of the entire neural network for a given input can now be efficiently computed by propagating the results of each layer through the network. This process is called *forward propagation*. To train the network, another process called *back-propagation* [33] is used to compute the gradients of a chosen metric, called the loss function. The loss function measures the difference between actual and desired output for a given input. A common loss function is the Mean Squared Error (MSE), $\text{MSE}(\hat{y}, y) = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$ [34]. Using the chain rule, the gradients of the loss function can be computed by

$$\frac{\partial L}{\partial w_{ij}^{(l)}} = \frac{\partial L}{\partial a_j^{(l)}} \frac{\partial a_j^{(l)}}{\partial z_{ij}^{(l)}} \frac{\partial z_{ij}^{(l)}}{\partial w_{ij}^{(l)}}. \quad (2.13)$$

The gradients now allow the usage of gradient descent [35] to update the weights and biases to lower the loss. A very basic approach to adjusting the weights is given by

$$w_{ij}^{(l)} \leftarrow w_{ij}^{(l)} - \eta \frac{\partial L}{\partial w_{ij}^{(l)}} \quad (2.14)$$

with the step size or learning rate η . In most cases, some variant of gradient descent is used, for example, Adaptive Moment Estimation (ADAM) [36].

A common problem in machine learning occurs when the model fits too closely to its training data. This behavior often appears when a model is too complex, which is called overfitting [37]. The model is usually never trained on all available data to prevent this. The data is split into two parts, called training and validation sets. The model is trained using the training set and validated with the validation set. Gradient descent is only applied to the training loss, never the validation loss. When both losses are similar in value, the model can generalize well, meaning it can correctly react to data that is not in the training data. If the validation loss is significantly higher than the training loss, overfitting occurs, and the model cannot generalize.

2.2.2 Diffusion models

In machine learning, there are generally two types of models, discriminative and generative [38]. They are not mutually exclusive, but especially less complex models can be attributed to either one of these labels. Discriminative models can be used to evaluate

the value of a variable given an observation. They do not generate new data but rather output a conditional probability, often used to classify data or to determine the relationship between variables as a regression problem. Generative models, on the other hand, are used to create new data artificially. The key concept is to learn the underlying distribution and generate random instances from this learned distribution. Diffusion models [39], specifically Denoising Diffusion Probabilistic Models (DDPMs) [9], are generative models that use noise to learn this distribution. Image generation is the most common use case with popular models like Stable Diffusion [40] or DALL-E 2 [41]. In general, any kind of data distribution can be learned, like natural language [42] or audio samples [43].

Algorithm 4 DDPM Training [9]

```

1: repeat
2:    $\mathbf{p}_0 \sim q(\mathbf{p}_0)$ 
3:    $t \sim \text{Uniform}(\{1, \dots, T\})$ 
4:    $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
5:   Take gradient descent step on
        $\nabla_{\theta} \|\epsilon - \epsilon_{\theta}(\sqrt{\alpha_t}\mathbf{p}_0 + \sqrt{1 - \alpha_t}\epsilon, t)\|^2$ 
6: until converged

```

The model architecture for a basic diffusion model is equivalent to a MLP, as described in Sec. 2.2.1. The process of learning and sampling, however, is different from that of a basic neural network. The general idea of a diffusion model is to generate new, synthetic data from randomly initialized noise. The network is trained to predict the added noise from iteratively noisier data to achieve this. In training, a forward process Alg. 4 is used, which is formulated as a Markov chain [44]. The noised data $\mathbf{p}_{t \neq 0}$ is computed from the original data $\mathbf{p}_0$ by

$$\mathbf{p}_t = \sqrt{\alpha_t}\mathbf{p}_0 + \sqrt{1 - \alpha_t}\epsilon \quad \forall t \neq 0 \quad (2.15)$$

with $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$. A noise schedule is used to create increasingly noisy data. It ensures $\mathbb{E}(\mathbf{p}_{i+1}) \leq \mathbb{E}(\mathbf{p}_i)$ and $\text{Var}(\mathbf{p}_{i+1}) \leq \text{Var}(\mathbf{p}_i)$. There are a few variants for a noise schedule,

the most basic one being the linear schedule [45]:

$$\begin{aligned}\beta_t &= \beta_{\text{start}} + t \frac{\beta_{\text{end}} - \beta_{\text{start}}}{N} \\ \alpha_t &= \prod_{T=1}^t (1 - \beta_T)\end{aligned}\tag{2.16}$$

The total steps N can vary depending on the type of data. In each training epoch, a gradient descent step is performed for uniformly sampled, discrete time steps t . The algorithm Alg. 4 uses the mean squared error to compute the gradient, but other loss functions are also viable. The time step indicates the amount of noise added to the original data and, as such, is added to the input of the diffusion model alongside the original data. Given $\mathbf{p}_t \in \mathbb{R}^m$, the input layer therefore has as size of $d_{\text{in}} = m + 1$, while the output omits t with $d_{\text{out}} = m$, as indicated in Fig. 2.9. With this training procedure, the model should be able to predict $\mathbf{p}_0$ from any element from the sequence $(\mathbf{p}_1, 1), (\mathbf{p}_2, 2), \dots, (\mathbf{p}_n, T)$ as input.

A new sample can be extracted from the model via the reverse diffusion process by initializing Gaussian noise $\mathbf{p}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and using it as input to the trained model. The output $\epsilon_\theta(\mathbf{p}_t, t)$ is then used to iteratively remove the noise from $\mathbf{p}_t$ according to line 4 in Alg. 5. This is done by starting with $t = T$ and decreasing it until reaching 1, at which point the output should resemble a sample from the original distribution.

With this basic approach, the model can only sample from **one** distribution. In many cases, the desired distribution can change depending on some conditions. In the use case of generating images, this could, for example, be the content or the shape of an object in the image. To differentiate between those different distributions, new values, called conditioning $p_{\text{condition}} \in \mathbb{R}^c$, are added to the model's input [46][47]. The input size changes to $d_{\text{in}} = m + c + 1$. The output dimension, however, stays the same.

When utilizing conditioning in a diffusion model, the conditioning in the training data has to be distributed over the entire spectrum that will later be sampled. If this is not fulfilled, it could lead to undesirable effects like hallucinations. A model is hallucinating when its output is incorrect or nonsensical [48][49].

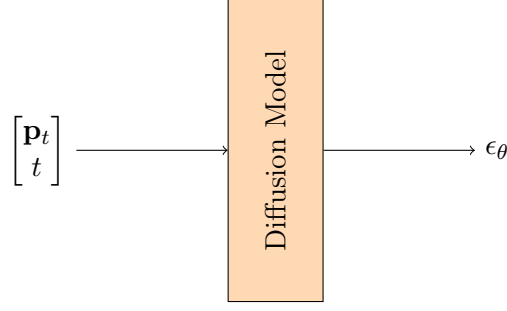


Figure 2.9: Single step of a DDPM with the input and output vector

Algorithm 5 DDPM Sampling [9]

```

1:  $\mathbf{p}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
2: for  $t = T, \dots, 1$  do
3:    $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$  if  $t > 1$ , else  $\mathbf{z} = \mathbf{0}$ 
4:    $\mathbf{p}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left( \mathbf{p}_t - \frac{1-\alpha_t}{\sqrt{1-\alpha_t}} \epsilon_\theta(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$ 
5: return  $\mathbf{p}_0$ 

```

2.3 Related Work

The method proposed by Pedrosa *et al.* [50] combines data-driven insights with model-based techniques for autonomous driving applications, which results in a maneuver automaton. It leverages the symmetry in vehicle dynamics to generalize motion patterns in human driving data. Motion primitives are clustered using the k-means clustering algorithm [51]. These clusters define nodes in the automation based on data-derived probabilities. Motion primitives are connected via an optimal control formulation, minimizing time cost.

Camci *et al.* [52] propose a learning-based framework for planning swift maneuvers of quadrotors. The approach focuses on building a motion primitive library by using Reinforcement Learning (RL) to explore and optimize Bézier curve-based control inputs. The training data is comprised of simulations and real experiments. A control authority mechanism selects the pre-learned motion primitives if quick maneuvers are needed.

A framework is proposed by Li *et al.* [53], that unifies Dynamic Movement Primitives (DMPs) and Probabilistic Movement Primitives (ProMPs) for generating smooth and probabilistically defined trajectories. DMPs allows for real-time replanning via attractor dynamics, which results in smooth trajectories, where parameters like speed can be changed without redesigning the system. ProMPs encodes trajectory variability by

modeling the uncertainty contained in a learned distribution. The combined framework called Probabilistic Dynamic Movement Primitives (ProDMPs), combines both the dynamic adaptability of DMPs and the statistical modeling of ProMPs. The approach focuses on its replanning aspect, which outputs smooth trajectories and is conditioned on sensory input.

In contrast to these works, the approach taken in this thesis aims to generate complete motion primitive datasets that are conditioned on a problem representation and later used by conventional kinodynamic motion planning algorithms. A further distinction is the use of DDPMs, which none of the related works presented use.

3 Approach

The main objective of this thesis is to create a diffusion model capable of generating motion primitives that are adaptive to arbitrary problem instances. This chapter discusses the approach to creating this model, including all processes needed to obtain the training data.

3.1 Dataset Creation

To be able to train a diffusion model to generate motion primitives specific to a problem instance, a dataset that represents this distribution is required. To generate such a dataset, db-A* is used to solve a specific problem with randomly generated motion primitives. This process is repeated several times to ensure that the random choice of motion primitives has no major influence on the general distribution of the solutions. Afterwards, all solutions are inspected. A solution consists of n_{MP} motion primitives, which are then extracted by their actions $\mathbf{u}$ and states $\mathbf{q}$. The cost and quantity of a given motion primitive are extracted as well, as this can be deduced from the solution. This information is then stored in a single dataset. This dataset can now be used to train a diffusion model.

3.2 Randomly Generating Problem Instances

To effectively generate a dataset for a model that is able to sample motion primitives for different problem instances, a large amount of different problem instances is needed. Creating them by hand is tedious and prone to introducing a bias in the instances themselves. Therefore, an automated approach to generating random problem instances is favorable.

This automated approach uses the problem instance definition presented in Eq. 2.9

and formulated in Alg. 6. The extent of the free workspace can be randomly set by

$$\begin{aligned} x_{\text{lb}} &= y_{\text{lb}} = 0 \\ x_{\text{ub}}, y_{\text{ub}} &\sim \mathcal{U} \in \{\mathcal{W}_{\text{ub},\text{min}}, \mathcal{W}_{\text{ub},\text{max}}\} \end{aligned} \tag{3.1}$$

with $\mathcal{W}_{\text{ub},\text{min}}$ and $\mathcal{W}_{\text{ub},\text{max}}$ being the bounds for the environment size to generate in. The next step in generating a random problem instance is the creation of the obstacles. There is the option to randomly place randomly sized obstacles in the workspace $\mathcal{W}$. This can, however, lead to a disconnected free workspace $\mathcal{W}_{\text{free}}$, which in return can lead to problem instances with no valid solution. Additionally, even a completely connected $\mathcal{W}_{\text{free}}$ does not guarantee the existence of a valid solution. A narrow passage can be too small for the robot to pass through. To avoid this, the obstacles are to be placed on a grid with grid spacing d_{grid} .

To keep track of all possible obstacle locations, the grid is initialized as a graph G (see Alg. 6, line 5), where every node is connected to all neighbor nodes, horizontal, vertical, and diagonal. Every obstacle will be a square of size d_{grid} . To place an obstacle, a random node N from G will be selected. If $G \setminus N$ is still fully connected, the obstacle can be placed at N (line 8). A user-set variable determines the percentage of the environment that is blocked with P_{obstacle} (line 3).

The final step in the instance generation is the selection of $\mathbf{q}_{\text{start}}$ and $\mathbf{q}_{\text{goal}}$. They could be randomly sampled from $\mathcal{W}_{\text{free}}$, but this could result in short distances between start and goal, in which the general environment has no great influence over the motion primitive distribution, which is unfavorable. Instead, the graph G calculates the shortest path between all pairs of nodes using Dijkstra's algorithm [54]. The solution with the longest path is selected, and the start and goal are extracted to be used as $\mathbf{q}_{\text{start}}$ and $\mathbf{q}_{\text{goal}}$ (line 12). The angle of the configurations is determined by the direction of the second and second to last node in the path, respectively. The general outline of this is shown in Alg. 6.

Algorithm 6 Random problem instance generation

Input: $\mathcal{W}_{\text{ub,min}}, \mathcal{W}_{\text{ub,max}}, P_{\text{obstacle}}, d_{\text{grid}}$

- 1: $x_{\text{ub}}, y_{\text{ub}} \sim \mathcal{U}\{\mathcal{W}_{\text{ub,min}}, \mathcal{W}_{\text{ub,max}}\}$
- 2: $P_{\text{obstacle}} \sim \mathcal{U}[W_{\text{blocked,min}}, W_{\text{blocked,max}}]$
- 3: $n_{\text{obstacle}} = P_{\text{obstacle}} \frac{x_{\text{ub}} y_{\text{ub}}}{d_{\text{grid}}^2}$
- 4: $\mathcal{W}_{\text{blocked}} = \{\}$
- 5: $G = \text{INITIALIZEGRAPH}(x_{\text{ub}}, y_{\text{ub}}, d_{\text{grid}})$
- 6: **while** $\text{SIZE}(\mathcal{W}_{\text{blocked}}) \neq n_{\text{obstacles}}$ **do**
- 7: $N \leftarrow \text{SELECTRANDOMNODE}(G)$
- 8: **if** $\text{ISGRAPHCONNECTED}(G \setminus N)$ **then**
- 9: $G = G \setminus N$
- 10: $R = \text{CREATERECTANGLE}(N.x, N.y, d_{\text{grid}}, d_{\text{grid}})$
- 11: $\text{ADDOBSTACLE}(\mathcal{W}_{\text{blocked}}, R)$
- 12: $\mathbf{q}_{\text{start}}, \mathbf{q}_{\text{goal}} \leftarrow \text{GETLONGESTPATH}(G)$
- 13: **return** $x_{\text{ub}}, y_{\text{ub}}, \mathcal{W}_{\text{blocked}}, \mathbf{q}_{\text{start}}, \mathbf{q}_{\text{goal}}$

Two examples of problem instances this algorithm generates are shown in Fig. 3.1. The start and goal configurations can be seen as green and red arrows. They show that selecting the longest path possible in a given problem instance is done correctly. In Fig. 3.1 (left) there exist multiple equally long solutions, in this case one is chosen at random. The input parameters for the generation are largely the same for both, with the environment size fixed with $\mathcal{W}_{\text{ub,min}} = \mathcal{W}_{\text{ub,max}} = 6$ and the grid size at $d_{\text{grid}} = 1$. The obstacle percentage is set as $P_{\text{obstacle}} = 0.3$ and $P_{\text{obstacle}} = 0.6$ respectively. As can be seen, the generated problems are relatively diverse, even though the limitation to only rectangular obstacles on a grid with a fixed size certainly influences the complexity of the problem instances. They are, however, quickly generated, and a solution, depending on the robot dynamics and the grid size, is almost guaranteed.

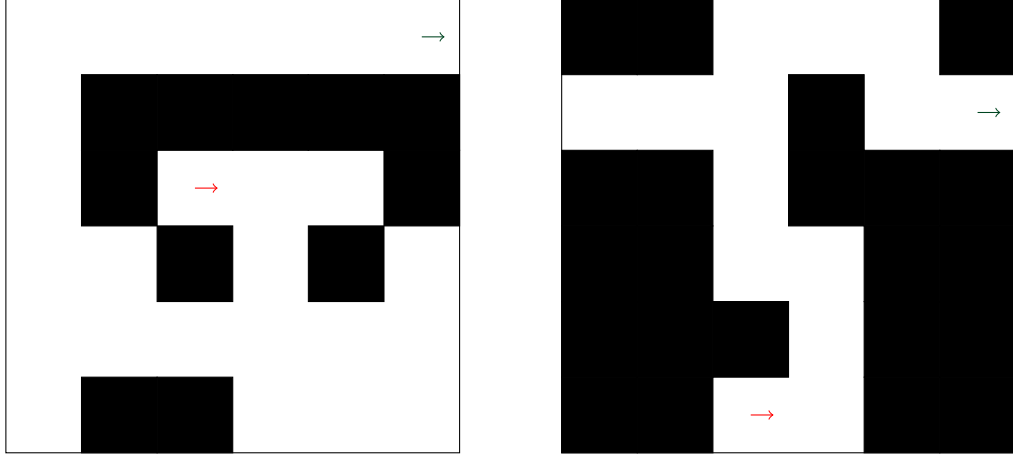


Figure 3.1: Two randomly generated problem instances with different obstacle densities ($\mathcal{W}_{\text{ub},\text{min}} = \mathcal{W}_{\text{ub},\text{max}} = 6$, $d_{\text{grid}} = 1$, $P_{\text{obstacle}} \in \{0.3 \text{ (left)}, 0.6 \text{ (right)}\}$)

3.3 Training a Basic Model

As a first approach, the task is simplified to a fixed problem instance. To generate the dataset for this first approach, two problem instances are chosen. The simpler *alcove* problem and the more complex *bugtrap* problem. The alcove problem, shown in Fig. 2.6, only requires the robot to make a right turn out of the alcove. This should intuitively lead to a motion primitive distribution with mostly right turns. The bugtrap problem, as shown in Fig. 2.7, however, requires a wide variety of motion primitives since two equal solution pathways are possible. Examples solutions for both are included in the figures referenced.

Two separate datasets are created, one for each problem instance. For each dataset, a model is trained using the layout that was discussed in Sec. 3.3.2. This approach does not utilize conditioning to learn multiple distributions. This is done to avoid complexity and verify the performance in a simpler setting. Combining multiple problem instances via conditioning in a single model will be discussed in detail in Sec. 3.4.5.

3.3.1 Learning Dynamics

A motion primitive, as described in Sec. 2.1.4, is a tuple of actions u and states q . A robot, for example, the unicycle as described in Sec. 2.1.3, is unable to move to any free location arbitrarily, and as such, valid motion primitives will follow a specific distribution. As discussed in Sec. 2.2.2, a diffusion model can learn any distribution, given a sizable

dataset and the appropriate complexity of the model. As such, training on a dataset of valid motion primitives should be able to then generate new, valid motion primitives. The required dataset is generated by sampling random actions $s, \phi \in [-0.5, 0.5]$ and then performing n successive Euler steps with n being the motion primitive length and dt being the discrete time step.

$$\mathbf{q}_i = \begin{pmatrix} x \\ y \\ \theta \end{pmatrix}, \quad \mathbf{u}_i = \begin{pmatrix} s \\ \phi \end{pmatrix} \quad (3.2)$$

$$\mathbf{q}_{i+1}(\mathbf{q}_i, \mathbf{u}_i) = \begin{pmatrix} \cos(\phi)sdt \\ \sin(\phi)sdt \\ \theta dt \end{pmatrix} \quad (3.3)$$

$$\mathcal{M} = \{[\mathbf{q}_0, \mathbf{q}_1, \dots, \mathbf{q}_n], [\mathbf{u}_0, \mathbf{u}_1, \dots, \mathbf{u}_{n-1}]\} \quad (3.4)$$

A basic diffusion model has a fixed input and output length, and therefore, the length n of all motion primitives in the training dataset is required to be fixed. With $n = 5$ and $dt = 0.1$ and no further conditioning, the input and output size of the diffusion model are set to

$$d_{\text{out}} = 3(n + 1) + 2n = 28 \quad (3.5)$$

$$d_{\text{in}} = d_{\text{out}} + 1 = 29 \quad (3.6)$$

$$\mathbf{p}_0 \in \mathbb{R}^{d_{\text{out}}}. \quad (3.7)$$

The input vector $\mathbf{p}_{\text{in}} \in \mathbb{R}^{d_{\text{in}}}$ is constructed as follows:

$$\mathbf{p}_{\text{in}} = \left[\mathbf{q}_0^\top \quad \dots \quad \mathbf{q}_5^\top \quad \mathbf{u}_0^\top \quad \dots \quad \mathbf{u}_4^\top \mid t \right]^\top = \left[\mathbf{p}_0^\top \mid t \right]^\top \quad (3.8)$$

Training the model is done as described in Sec. 2.2.2 using the dataset described in Sec. 3.1. The model is trained to predict the noise level set by t added to $\mathbf{p}_0$. When sampling from the model, the output should be a valid motion primitive

$$\mathcal{M}^{\text{pred}} = \left\{ \left[\mathbf{q}_0^{\text{pred}}, \mathbf{q}_1^{\text{pred}}, \dots, \mathbf{q}_n^{\text{pred}} \right], \left[\mathbf{u}_0^{\text{pred}}, \mathbf{u}_1^{\text{pred}}, \dots, \mathbf{u}_{n-1}^{\text{pred}} \right] \right\}. \quad (3.9)$$

To validate this output and by extent the performance of the model, the actions of the sampled motion primitives $\mathbf{u}_i^{\text{pred}}$ are used to compute the states $\mathbf{q}_i^{\text{calc}}$, that would be

reached if the actions were applied to the robot. If the diffusion model correctly learned the dynamics f of the robot, the predicted states $\mathbf{q}_i^{\text{pred}}$ and the states $\mathbf{q}_i^{\text{calc}}$ calculated from the predicted actions $\mathbf{u}_i^{\text{pred}}$ should be almost identical. Using the predicted states of the sampled motion primitives $\mathbf{q}_i^{\text{pred}}$, the mean square error is given by

$$\frac{1}{n} \sum_{i=0}^n \left(\mathbf{q}_i^{\text{calc}} - \mathbf{q}_i^{\text{pred}} \right)^2. \quad (3.10)$$

The error does not indicate whether the dataset and sampling distribution match. It only indicates whether the diffusion model is able to sample valid motion primitives. The distribution matching is measured by the metric used in the training process.

3.3.2 Learning Actions Distribution

The states $\mathbf{q}^{\text{calc}}$ can be easily computed from the actions $\mathbf{u}^{\text{pred}}$. Since the dynamics of a first-order-unicycle are translation- and rotation-invariant, all motion primitives can be translated and rotated to start from any valid $\mathbf{q}_{\text{start}}$. Information about the starting state is, therefore, redundant. With f_d being a deterministic function, it follows from

$$\begin{aligned} \mathbf{u}^{\text{pred}} &\sim \mathbf{u}^{\text{real}} \\ \Rightarrow f(\mathbf{u}^{\text{pred}}) &\sim f_d(\mathbf{u}^{\text{real}}) \\ \Rightarrow \mathbf{q}^{\text{calc}} &\sim \mathbf{q}^{\text{real}} \end{aligned} \quad (3.11)$$

The states $\mathbf{q}$ can, therefore, be excluded almost completely from the dataset and the training process. db-A* considers only translation invariant dynamics. This is expanded upon in [55], but this extension is not considered in this work. For this reason, the starting angle θ_0 will remain in the dataset. This significantly reduces the model's complexity. All models presented, excluding the first dynamics model, do not attempt to learn the dynamics f .

With the same parameters as in 3.3.1, the model dimension and the input vector are now defined as

$$d_{\text{out}} = 2n + 1 = 11 \quad (3.12)$$

$$d_{\text{in}} = d_{\text{out}} + 1 = 12 \quad (3.13)$$

$$\mathbf{p}_{\text{in}} = \left[\mathbf{u}_0^\top \quad \dots \quad \mathbf{u}_4^\top \quad \theta_0 \mid t \right]^\top = \left[\mathbf{p}_0^\top \mid t \right]^\top \quad (3.14)$$

The model now only has to learn one distribution, not two separate distributions that are linked by f .

3.4 Extensions

This section presents possible extensions to the basic model shown in Sec. 3.3.2, which includes conditioning, as well as modification and variation of the dataset.

3.4.1 Conditioning on Probability

The dataset generated in Sec. 3.1 grows linearly in size with the number of solutions generated for a problem instance, as every motion primitive extracted is treated as a single data point. Some motion primitives, however, might occur more often than others, signifying their importance in this specific problem instance. Extracting only the unique motion primitive without any notion of the number of appearances would lead to a loss of information. The motion primitives should, therefore, be weighted accordingly. When modifying the dataset, in this case, for performance reasons, the distribution should not be changed significantly. To achieve this, the extraction process counts the occurrences of each unique motion primitive over all solutions. This information is added to the dataset. The number of occurrences n_i of a motion primitive $\mathcal{M}_i$ could be used directly to condition the diffusion model, but this would lead to complications when trying to sample from the model, as $\max n_i$ is dependent on the dataset and the specific problem. Generating samples from this would lead to lost information, as uniform sampling would either not use information from longer solutions or sampling for n_i where there is only sparse training data. To solve this problem, instead of using n_i as a condition, the relative probability

$$P_{\text{rel}}(n_i) \in [0, 1] = \frac{n_i}{\max n} \quad (3.15)$$

is used. This allows to easily sample from the distribution. This can be done with $\mathbf{p}_{\text{condition}} \in \mathcal{U}(0, 1)$. Another option is to weigh motion primitives with larger P_{rel} even higher by, for example, sampling from a distribution that is given by Cumulative Distribution Function (CDF) $= x^{10} \quad \forall x \in [0, 1]$. This, however, requires enough data points with high P_{rel} , to avoid side-effects like hallucinations, as discussed in Sec. 2.2.2.

The training process remains largely the same, merely the input dimension

$$d_{\text{in}} = d_{\text{out}} + d_{\text{cond}} + 1 = 10 + 1 + 1 = 12 \quad (3.16)$$

has to be adjusted. The conditioning P_{rel} is computed during runtime and not included in the dataset itself to keep maximum flexibility for further adjustments. The input vector is now constructed as

$$\mathbf{p}_{\text{in}} = \left[\mathbf{u}_0^\top \quad \dots \quad \mathbf{u}_4^\top \quad \cos \theta_0 \quad \sin \theta_0 \mid P_{\text{rel}} \mid t \right]^\top = \left[\mathbf{p}_0^\top \mid \mathbf{p}_{\text{condition}}^\top \mid t \right]^\top. \quad (3.17)$$

3.4.2 Conditioning on Location

The approach in Sec. 3.4.1 is a valid method to further separate the distributions of the motion primitive dataset. There are, however, some problems regarding the sampling, as there is no guarantee for a good data coverage over all P_{rel} . On the contrary, it is entirely possible that for most motion primitives $P_{\text{rel}} \rightarrow 0$. As the premise of introducing P_{rel} is to sample for more important motion primitives, this would defeat its purpose. Another, more uniformly spread parameter of a motion primitive in a solution is its relative location $L_{\text{rel}} \in [0, 1]$. Given a solution, consisting of n_{MP} motion primitives, where n_k is the length of the k -th motion primitive, with

$$n_{\text{dt}} = \sum_{k=1}^{n_{\text{MP}}} n_k \quad (3.18)$$

discrete time steps, the relative location is given by

$$L_{\text{rel}}(\mathcal{M}_j) = \frac{n_{\text{dt}}}{1 + \sum_{k=1}^{j-1} n_k}. \quad (3.19)$$

As every solution has motion primitives uniformly distributed from $\mathbf{q}_{\text{start}}$ to $\mathbf{q}_{\text{goal}}$, the dataset will have a completely uniform distribution of L_{rel} . This applies regardless of solution length. Therefore, sampling with $L_{\text{rel}} \sim \mathcal{U} \in [0, 1]$ should lead to more stable results and could be a good alternative to Sec. 3.4.1. This approach does not explicitly weigh motion primitives based on their occurrence, which could arguably lead to worse performance. This, however, has to be inspected in practice. The resulting input vector is constructed by setting

$$\mathbf{p}_{\text{condition}} = \left[L_{\text{rel}} \right]. \quad (3.20)$$

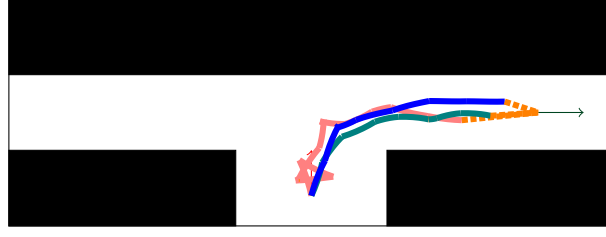


Figure 3.2: Three example solutions for the alcove problem for different discontinuity bounds - $\delta \in \{0.3$ (*blue*), 0.5 (*teal*), 0.7 (*red*) $\}$, discontinuity *orange*

3.4.3 Conditioning on δ

As mentioned in 2.1.5, db-A* has a user-defined parameter δ , which sets the maximum allowed discontinuity between motion primitives. Intuitively, a smaller delta should lead to a more well-defined distribution for a given problem instance, as the different solutions that make up the dataset are more constrained. To investigate this, the dataset creation has to be expanded. Instead of only solving the problem instances with $\delta = 0.5$, multiple values for delta are used. For this purpose, $\delta \in \{0.3, 0.5, 0.7\}$ is chosen. For each δ , an example solution is shown in Fig. 3.2. The discontinuity bound, shown in orange, clearly grows smaller with a decreasing $\delta \in \{0.3$ (*blue*), 0.5 (*teal*), 0.7 (*red*) $\}$ and the resulting trajectories get smoother. Choosing a lower δ is not practical, as only the base motion primitive dataset is used without optimized primitives being added. The δ used for a given solution is recorded and added to the dataset. The occurrences of a unique motion primitive are now counted for each δ since different distributions are expected. Analogous to Sec. 3.4.1, δ is added to the conditioning.

$$\mathbf{p}_{\text{condition}} = \begin{bmatrix} P_{\text{rel}} \vee L_{\text{rel}} & \delta \end{bmatrix}^{\top}. \quad (3.21)$$

3.4.4 Alternative Representation of θ_0

The current representation of the starting angle θ_0 leads to some problems regarding the evaluation of the model with its cost function. Most cost functions like the MSE do not consider circular spaces like $\theta_0 \in S^1$. The maximum error for two angles θ_0^1 and θ_0^2 is reached with $\text{MSE}(-\pi, \pi) = 2\pi^2$, even though they represent the same angle. The learned distribution with MSE will most likely represent the actual distribution better at $|\theta_0| \approx 0$ and worse at $|\theta_0| \rightarrow \pi$.

There are some possible approaches to solve this problem. The first option is using a

cost function that accurately considers the circular properties. By separately considering the error for $\mathbf{u}$ as the traditional MSE and the error for θ_0 as the absolute angle difference, for example, computed with

$$d_{\text{abs}}(\theta_0^1, \theta_0^2) = \text{abs} \left(\arctan2 \left(\sin(\theta_0^2 - \theta_0^1), \cos(\theta_0^2 - \theta_0^1) \right) \right). \quad (3.22)$$

This metric accurately describes the error for the angle, but it can introduce a new problem, as now

$$d_{\text{abs}}(\theta_0^1, \theta_0^2) = \begin{cases} 0, & \text{if } \theta_0^2 = \theta_0^1 + 2n\pi \quad \forall n \in \mathbb{Z} \\ > 0, & \text{otherwise.} \end{cases} \quad (3.23)$$

Given a specific value for θ_0^1 , there are now an infinite amount of values for θ_0^2 , to have an error of 0, which can lead to instability in the diffusion model, as it could try to model each data point to another, individual angle value.

A third approach is to use an alternative representation for the angle. As presented in [56], the usage of a quaternion or a rotation matrix is the most promising. The main goal of the alternative representation is to represent θ_0 by (mostly) linear distributed values and, by extent, remove the circular behavior. The quaternion q with

$$\begin{aligned} q(\theta_0) &= q_w + q_x \mathbf{i} + q_y \mathbf{j} + q_z \mathbf{k} \\ \mathbf{i}^2 &= \mathbf{j}^2 = \mathbf{k}^2 = \mathbf{ijk} = -1 \\ 1 &= \sqrt{q_w^2 + q_x^2 + q_y^2 + q_z^2} \\ q_w &= \cos \frac{\theta_0}{2} \quad q_x = q_y = 0 \quad q_z = \sin \frac{\theta_0}{2} \end{aligned} \quad (3.24)$$

(not to be confused with the state $\mathbf{q}$) represents θ_0 with four scalars, although $q_x = q_y = 0$ do not have to be learned. A quaternion can parameterize 3-dimensional rotations, but in this instance, the rotation is only in the xy -plane. A valid quaternion needs to be normalized, as per line 3 in Eq. 3.24. As the sampled output of the diffusion model will be noisy, a normalized output can not be expected. Normalization and conversion to θ_0 is therefore done as post-processing.

The issue regarding the cost function is still present in this representation. As in the

representation, the problem occurs at the bounds:

$$\begin{aligned} q(\theta_0^1 = -\pi) &= -k \\ q(\theta_0^2 = \pi) &= k \end{aligned} \tag{3.25}$$

These angles lead to different representations and also to the largest error possible for $\theta_0 \in [-\pi, \pi]$ using MSE. Therefore this approach does not improve upon the original representation.

Another approach to represent θ_0 is using a rotation matrix. A rotation matrix is used to transform a vector by rotating it. A rotation matrix to rotate a vector $\mathbf{v}$ by θ_0 to $\mathbf{v}_{\text{rot}}$ is given by

$$\mathbf{R} = \begin{bmatrix} \cos \theta_0 & -\sin \theta_0 \\ \sin \theta_0 & \cos \theta_0 \end{bmatrix} \tag{3.26}$$

$$\mathbf{v}_{\text{rot}} = \mathbf{R}\mathbf{v} \tag{3.27}$$

The angle θ_0 represented by $\mathbf{R}$ can be calculated as post-processing by

$$\mathbf{v}_{\text{rot}} = \mathbf{R} \begin{bmatrix} 1 \\ 0 \end{bmatrix} \tag{3.28}$$

$$\theta_0 = \arctan2(\mathbf{v}_{\text{rot}}). \tag{3.29}$$

As the post-processing step only requires the first column of $\mathbf{R}$ and the matrix only consists of two unique values (with one negation), the parameters that need to be learned are

$$\Theta_0 = \mathbf{R} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} \cos \theta_0 \\ \sin \theta_0 \end{bmatrix}. \tag{3.30}$$

As can be seen, both the quaternion and Θ_0 lead to similar representations of θ_0 . The problem observed in both the original representation as well as the one using the quaternion does not exist in this approach, as

$$\Theta_0(\theta_0^1 = -\pi) = \Theta_0(\theta_0^2 = \pi) = \begin{bmatrix} 1 \\ 0 \end{bmatrix}. \tag{3.31}$$

With this representation, any two angles representing the same absolute angle lead to a MSE of 0, without the drawback of having infinite representations for the same angle.

However, using sin and cos leads to a non-linear representation. This should not lead to major issues since a larger absolute error between angles always leads to a larger error and vice versa. Using this approach, the input vector now results from changing

$$\mathbf{p}_0 = \begin{bmatrix} \mathbf{u}_0 & \dots & \mathbf{u}_4 & \cos \theta_0 & \sin \theta_0 \end{bmatrix}^\top. \quad (3.32)$$

3.4.5 Conditioning on Environment

All previous models are specific to one single problem instance. Switching to another requires the training of a completely new model, as there is currently no parameter to separate the distributions from different problem instances. A simple approach to this would be to assign a number to every problem instance in the dataset. This, however, would be quite limiting, as the previous models can only sample from the specific instance it was trained on. A new problem instance could not be sampled from, as it is out of the current distribution and there is no way of quantifying its relation to the problem instances present in the dataset. Therefore, different parameters have to be selected to parameterize the problem instances.

A few parameters can be easily extracted from the problem instance as defined in 2.9. To be able to integrate a parameter into the existing model, the amount and size of the selected parameters have to be constant over all environments, as the model architecture does not allow a resizing of the input or output.

The trivial choices for parameters, whose dimension stays constant and which are directly stated in the definition, are the corners of the rectangular environment given by $(x_{\text{lb}}, x_{\text{ub}})$ and $(y_{\text{lb}}, y_{\text{ub}})$, as well as the starting state $\mathbf{q}_{\text{start}}$ and goal state $\mathbf{q}_{\text{goal}}$.

Some further parameters can be calculated. The extents of the environment are given by

$$d_x = x_{\text{ub}} - x_{\text{lb}}, \quad d_y = y_{\text{ub}} - y_{\text{lb}}. \quad (3.33)$$

As the motion primitives are translation invariant, the relative extents can certainly be used instead of the corners, thus reducing the dimensionality. Instead of using the area

$$A(\mathcal{W}) = d_x d_y \quad (3.34)$$

is another option, although some information may be lost in this approach.

Parameters that describe or correlate with the obstacles should have an intuitively higher correlation to the distribution of motion primitives, as an (almost) empty environ-

ment does not require highly specialized movement. Even though the blocked workspace $\mathcal{W}_{\text{blocked}}$ is directly given by a set of parameters, the amount of parameters depends on the number of obstacles. Therefore, broader information has to be used. One option is the use of the blocked area

$$A(\mathcal{W}_{\text{blocked}}) = \sum_{i=1}^{n_R} w_i h_i + \sum_{j=1}^{n_c} \pi r_j^2. \quad (3.35)$$

Another option is the more well-bounded probability for hitting an obstacle when randomly sampling a point in $\mathcal{W}$.

$$P((x, y) \in \mathcal{W}_{\text{blocked}} \mid (x, y) \sim \mathcal{W}) = \frac{A(\mathcal{W}_{\text{blocked}})}{A(\mathcal{W})} \quad (3.36)$$

The start $\mathbf{q}_{\text{start}}$ and goal state $\mathbf{q}_{\text{goal}}$ certainly contain some information about the general direction of motion. Analogous to the extent of the environment, the absolute coordinates hold redundant information since the motion primitives are translation invariant and no states are sampled directly. Therefore, the relative state

$$\mathbf{q}_{\text{rel}} = \begin{bmatrix} x_{\text{goal}} - x_{\text{start}} \\ y_{\text{goal}} - y_{\text{start}} \\ d_{\text{abs}}(\theta_0^{\text{goal}}, \theta_0^{\text{start}}) \end{bmatrix}, \quad (3.37)$$

or parts of it can be used to reduce the dimensionality of the data.

All of the parameters mentioned above are possible ways of describing a problem instance. Their respective influence on the model's performance can only be estimated and has to be evaluated empirically.

3.4.6 Varying Motion Primitive Lengths

All previously discussed models and datasets are limited in one regard – the fixed length of the motion primitives. This is due to the architecture of a single model, which only has a fixed amount of input and output nodes. Being able to generate different length motion primitives can be an improvement. Longer motion primitives lead to fewer vertices in the graph which should improve the runtime, while shorter motion primitives are more flexible, especially in complex environments. Motion primitives in datasets for kinodynamic motion planning algorithms are usually in a certain range of lengths. Replicating this behavior with a diffusion model is something that should be explored.

A solution that contains different length motion primitives introduces a new distribution – the percentage of each length. It is, therefore, feasible to learn this distribution and predict which lengths should be used given a specific problem instance. This information can be used to query multiple models to generate the actual motion primitives. In short, instead of utilizing a more complex architecture, multiple diffusion models are used together. All models have to be trained and tuned separately. To simplify this process, only a small amount of different lengths are selected, for example, 5, 10, and 15. This leads to four different models, one to allocate the desired samples to the different lengths and one each to generate the actual motion primitives. This model will be referred to as the composite model.

4 Results

After discussing the different approaches to generating motion primitives, the different models will now be evaluated.

4.1 Setup

To measure the performance of the different models discussed in Cha. 3, a few metrics are of interest. The training and validation loss of a model can be observed in the training process and it should indicate whether the model is able to fit the distribution provided in the training data without overfitting, as explained in Sec. 2.2.1. This is the most direct way of measuring the model performance, but it is unclear if this metric has a major influence on the performance in a benchmark setting. A low validation loss shows that the model is capable of sampling from the distribution present in the data, but depending on the chosen parameters, this may not directly correlate to a superior performance of the sampled dataset.

The main goal of this thesis is to improve the motion primitive datasets, meaning the solution given by the algorithm should be of lower cost, and it should take db-A* less time to find a solution than with the original dataset. This duration only considers the algorithm presented in Alg. 2. Further necessary operations like loading of the dataset are excluded from the measured duration, as they can vary depending on the dataset size and format. The cost is defined as the time it would take to execute the solution. Another metric that can be observed is the success rate. Given a dataset of a limited size, the algorithm may be unable to find a solution to some more complex problems. If the generated datasets are more specific to the problem, a higher success rate can be expected. The quality of the solution is also of interest. This cost is measured in the amount of time the robot would take to execute the found solution.

As the different approaches presented in this thesis are in part designed for different setups, they will be compared in sequence, as they were presented in Cha. 3. All approaches, except the first, will be evaluated by running db-A* with their respective gen-

erated datasets. All results utilize the unicycle and its dynamics presented in Sec. 2.1.3. Unless stated otherwise, all models contain three fully connected hidden layers with 128 neurons with Rectified Linear Unit (ReLU) as the activation function. The epoch with the lowest validation loss will be used for sampling. The batch size will be set to 512, and 30 denoising steps will be used. The learning rate is also set to $1 \cdot 10^{-3}$ utilizing the optimizer ADAM. These hyperparameters were manually set and kept constant until fine-tuning for better comparability. The models are implemented in Python using PyTorch [57]. Each model will be queried to sample 100 motion primitives 20 times for any given problem instance, which are then used to run db-A*.

4.2 Models for Specific Problem Instances

To evaluate if the models are able to improve upon the original dataset, a baseline has to be established. The baseline is created by running db-A* with the dataset that is also used to train the models.

Learning Dynamics Equation

The model input and output layers are implemented as described in Sec. 3.3.1. In Fig. 4.1, the training and the validation loss are shown. The loss function used is the MSE. When calculating the error between the predicted states and the states that would be reached by applying the predicted actions, the error for 1000 samples given by

$$\frac{1}{n} \sum_{i=0}^n \left(\mathbf{q}_i^{\text{calc}} - \mathbf{q}_i^{\text{pred}} \right)^2. \quad (4.1)$$

is 0.0424 for the alcove model and 0.0432 for the bugtrap model. For benchmarking this model with db-A*, the predicted actions $\mathbf{u}^{\text{pred}}$ alongside the **calculated** states $\mathbf{q}^{\text{calc}}$ are used for the motion primitives of the dataset.

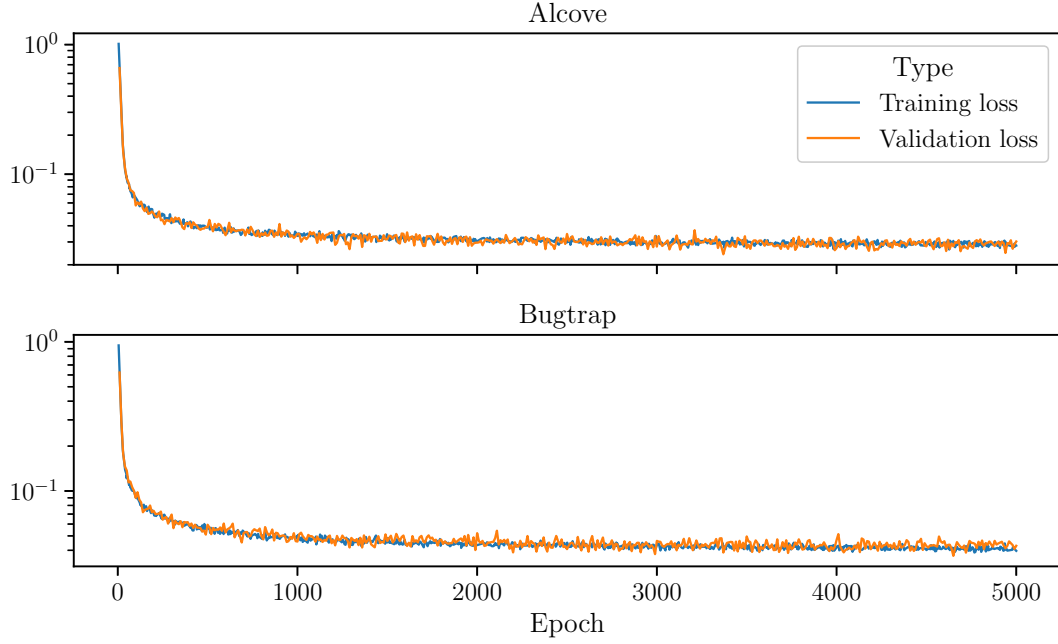


Figure 4.1: Training and validation loss of the dynamics model for the alcove and bugtrap problem

Learning Actions Distribution

The training process of the basic model for learning the action distribution is done similarly to the last model. The losses of this and all future models run similarly to the one shown in Fig. 4.1, with only the minimum achieved loss changing from model to model. For this reason, all remaining loss plots are shown in Appendix A. The final minimum losses for all models in this subsection are shown in Tab. 4.1. The models discussed in Sec. 3.4.1 and Sec. 3.4.2 introduce conditioning with either P_{rel} or L_{rel} . The training process, however, remains largely the same, as presented in Sec. 2.2.2. Their respective training and validation losses can be seen in Fig. A.2 and Fig. A.3.

The model that utilizes δ as conditioning requires an extended dataset, as the original one does not cover multiple values for δ . This is done by repeating the process for the creation with the different desired values and adding δ to the extracted data. In this case $\delta \in \{0.3, 0.5, 0.7\}$ is chosen. The training and validation loss are shown in Fig. A.4.

The last model addition proposed for a fixed environment is the alternative representation of θ_0 as $(\cos \theta_0, \sin \theta_0)$. As those values are not direct parts of the dataset, they

Table 4.1: Minimum losses for a fixed problem instance

Model	Instance	Training loss [10^{-2}]	Validation loss [10^{-2}]
Dynamics	Alcove	2.62	2.41
	Bugtrap	3.76	3.67
Basic	Alcove	4.90	4.35
	Bugtrap	6.63	6.37
P_{rel}	Alcove	4.23	3.79
	Bugtrap	6.58	5.99
L_{rel}	Alcove	3.50	3.26
	Bugtrap	5.74	5.37
δ, L_{rel}	Alcove	3.42	3.21
	Bugtrap	5.64	5.42
Θ_0, L_{rel}	Alcove	3.25	2.92
	Bugtrap	4.88	4.63

are calculated on runtime. They also have to be converted back to the original representation after sampling. As before, the training process is unaffected. The training and validation loss are shown in Fig. A.5.

In Tab. 4.1, the best achieved losses for each model are presented. Training and validation loss are both formatted to be multiplied by 10^{-2} . The lowest losses for each respective problem instance are marked in bold. The benchmark results of all models in this section and the baseline dataset are presented in Tab. 4.2. The success column shows the mean success rate over 20 trials. For duration and cost, the median is used. Additionally, for duration and success, the best value is indicated in bold for both instances. The overall best performance is achieved by the model utilizing Θ_0 and L_{rel} . The model utilizing only P_{rel} is able to solve the bugtrap problem significantly faster, it however fails to solve it consistently. Therefore, this data is shown in italics and not taken into account.

To better visualize the performance of the models compared to the baseline, the notion of regret for duration r_d and cost r_c is presented in boxplots. It is calculated by subtracting the median value of the baseline and then dividing by it. The baseline itself is also shown to present its performance distribution. The median value of the Baseline at $r_d = 0$ and $r_c = 0$ is marked as a red dotted line. For both r_d and r_c a lower value represents an improvement over the baseline.

The relative performance r_d and r_c of all models and the baseline for the alcove problem is shown in Fig. 4.2 on the left and right respectively. The dynamics model was not able

Table 4.2: Benchmark results for a fixed problem instance

Model	Instance	Success [%]	Duration [ms]	Cost [s]
Baseline	Alcove	94	20.13	4.00
	Bugtrap	100	490.83	21.00
Dynamics	Alcove	0	NaN	NaN
	Bugtrap	100	561.11	32.00
Basic	Alcove	95	17.05	3.50
	Bugtrap	100	506.06	30.00
P_{rel}	Alcove	60	7.35	3.50
	Bugtrap	100	483.34	19.50
L_{rel}	Alcove	100	12.34	3.00
	Bugtrap	100	492.45	19.00
δ, L_{rel}	Alcove	100	12.20	3.00
	Bugtrap	100	482.63	19.50
Θ_0, L_{rel}	Alcove	100	11.88	3.00
	Bugtrap	100	452.08	19.00

to find a solution for the alcove problem and is therefore not shown in this plot. The results for the bugtrap problem are presented in Fig. 4.3 in a similar fashion.

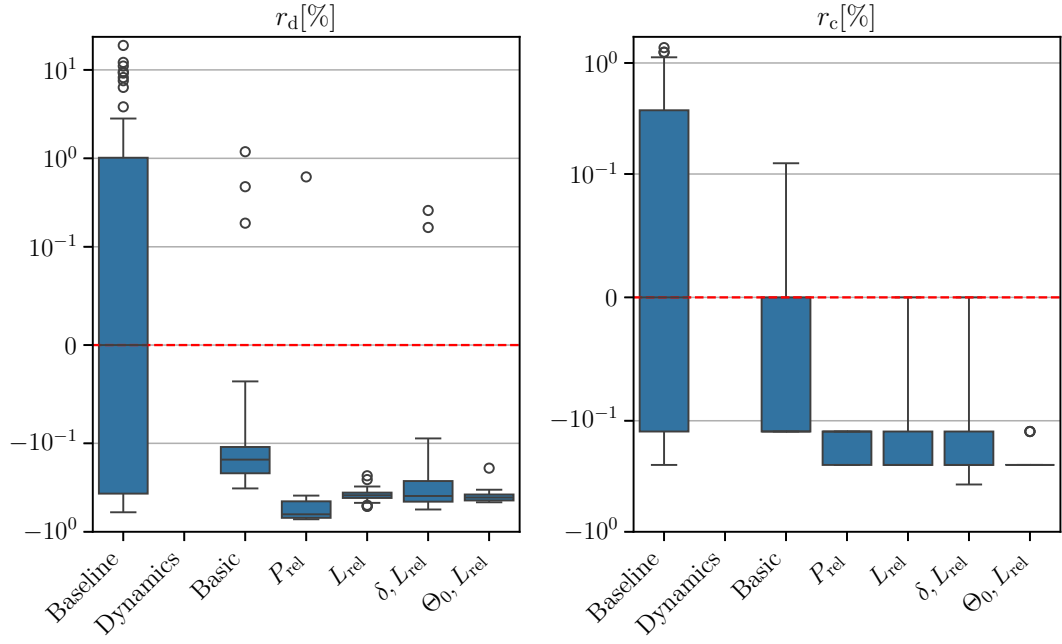


Figure 4.2: Notion of regret for duration and cost for the alcove problem

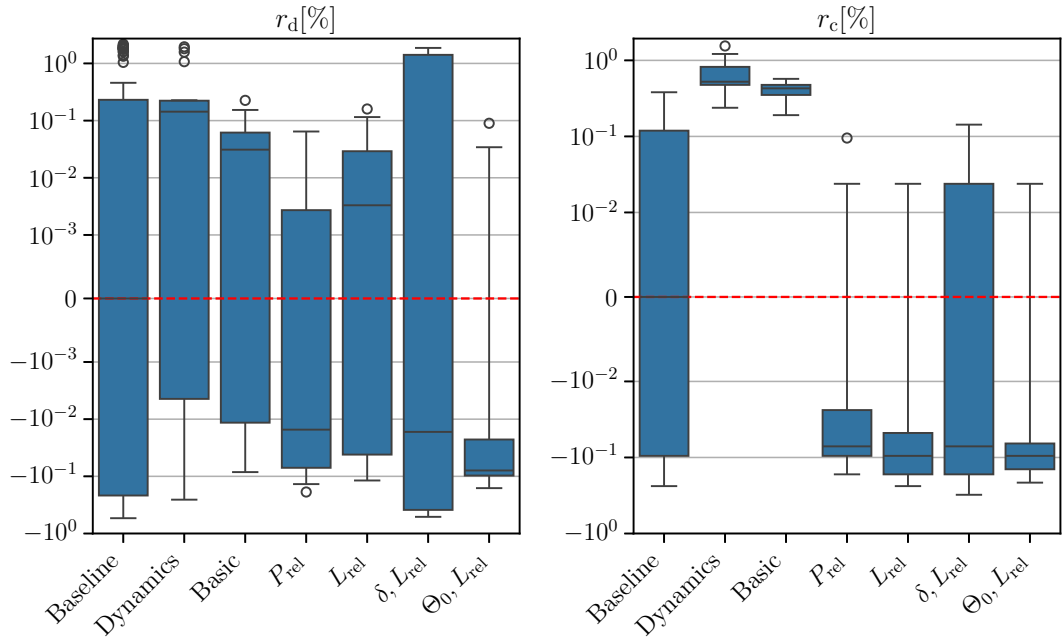


Figure 4.3: Notion of regret for duration and cost for the bugtrap problem

4.3 Models for Multiple Problem Instances

Utilizing the algorithm presented in Alg. 6, a dataset for models for multiple problem instances is created. It introduces the various conditioning parameters mentioned in Sec. 3.4.5.

4.3.1 Parameter Selection

To explore the different options to combine the parameters, multiple models have to be trained. Evaluating them based on the benchmark is too inefficient, therefore the validation loss will be used. Evaluating every possible combination of parameters is infeasible since the selected ten parameters result in 1024 possible models. For this reason, the search algorithm Optuna [58] in conjunction with the Asynchronous Successive Halving (ASHA) scheduler [59] is used. The search algorithm explores the different parameter combinations, while the scheduler allows for efficient parallel training by stopping a fraction of the worst-performing models at defined epochs. This way, not all models have to be trained for the same number of epochs.

The top four parameter configurations are selected from the search algorithm run. The selected models use the following parameters:

Table 4.3: Hyperparameter selection for the top four models found by Optuna

Model No.	P_{rel}	L_{rel}	$P(\mathcal{W}_{\text{blocked}})$	θ_0^{goal}
1	×	×	-	×
2	×	×	×	×
3	-	×	×	×
4	-	×	×	-

The top 15 models out of 128 tested utilize one of these configurations. No other parameters are present. Each model is benchmarked according to Fig. 4.4. As before, the number of trials is set to $n_t = 20$. The number of random problem instances is set to $n_p = 40$. Therefore, each model has 800 trials, summing up to a total of 3200 evaluation runs. The minimum losses are shown in Tab. 4.4. The median benchmark results and the notion of regret of all models is shown in Tab. 4.5 and Fig. 4.5 respectively.

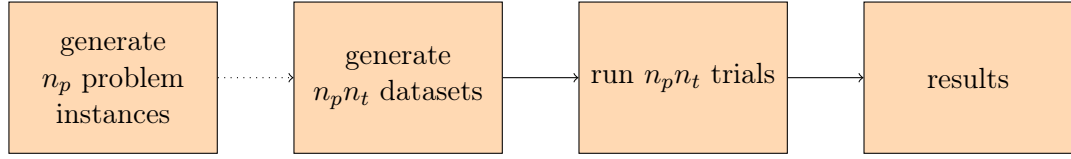


Figure 4.4: Pipeline benchmark

Table 4.4: Minimum losses of the top four models in the parameter selection for multiple problem instances

Model	Training loss [10^{-2}]	Validation loss [10^{-2}]
Model 1	5.82	5.53
Model 2	5.78	5.69
Model 3	5.61	5.47
Model 4	5.80	5.70

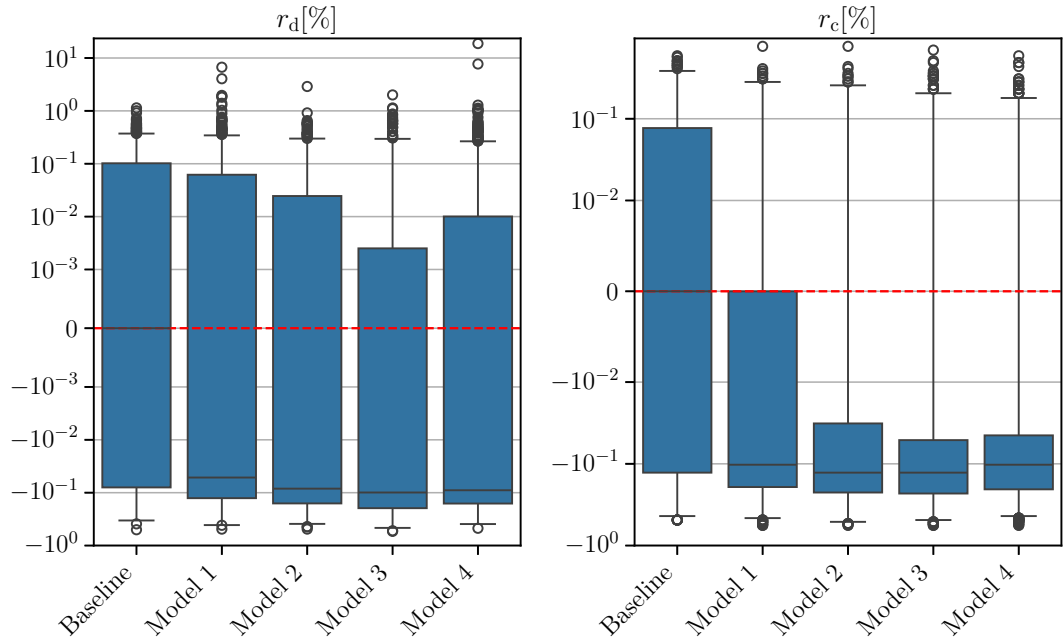


Figure 4.5: Notion of regret for duration and cost of the top four models in the parameter selection for multiple problem instances

Table 4.5: Benchmark results for the top four models in the parameter selection for multiple problem instances

Model	Success [%]	Duration [ms]	Cost [s]
Baseline	100	136.68	19.50
Model 1	100	129.61	17.50
Model 2	100	125.21	17.00
Model 3	99	123.12	17.00
Model 4	100	124.38	17.50

4.3.2 Fine-Tuning

Based on the results of the previous benchmark, the best-performing model is selected for further fine-tuning. By employing the same method as before for hyperparameter selection, the hidden layer dimensions and the learning rate are adjusted.

Table 4.6: Hyperparameter selection for the fine-tuned model 3 for multiple problem instances

Model	Hidden Layers	Node per hidden layer	Learning rate
Model 3	3	128	1×10^{-3}
Model 3 tuned	2	256	2×10^{-3}

Table 4.7: Minimum losses for the fine-tuned model 3 for multiple problem instances

Model	Training loss [10^{-2}]	Validation loss [10^{-2}]
Model 3	5.61	5.47
Model 3 tuned	5.21	5.13

Table 4.8: Benchmark results for the fine-tuned model 3 for multiple problem instances

Model	Success [%]	Duration [ms]	Cost [s]
Baseline	100	136.68	19.50
Model 3	99	123.12	17.00
Model 3 tuned	100	115.83	17.00

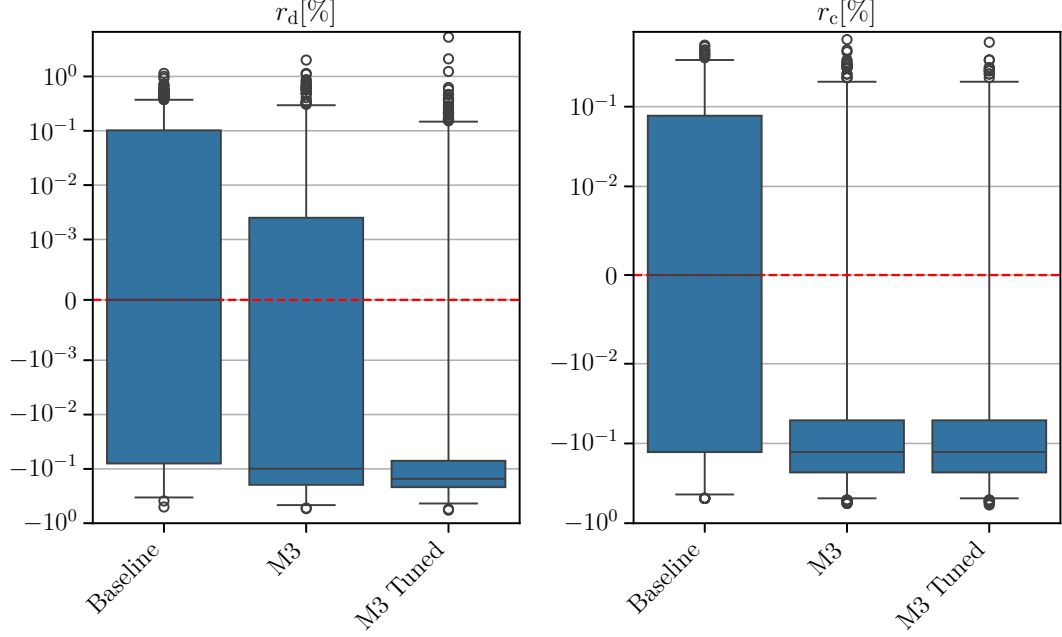


Figure 4.6: Notion of regret for duration and cost for the fine-tuned model 3 for multiple problem instances

4.3.3 Composite Model

The model proposed in Sec. 3.4.6 utilizes a new dataset, now including multiple-length motion primitives. As a reference, both the original baseline and the new multi-length baseline are included in the plots below. All four models that are part of the composite model are trained and optimized in the same manner as before. The resulting hyperparameters are shown in Tab. 4.9. As the composite model is made up of four diffusion models that are trained separately, the loss can only be shown for each component, which does not allow for a combined loss value. The losses achieved by each model are shown in Tab. 4.10.

Table 4.9: Hyperparameter selection for the composite model for multiple problem instances

Model	Hidden Layers	Nodes per hidden layer	Learning rate
Allocator	2	16	1×10^{-3}
Length 5	2	256	2×10^{-3}
Length 10	2	384	1×10^{-3}
Length 15	3	512	9×10^{-4}

Table 4.10: Minimum losses for the composite model for multiple problem instances

Model	Training loss [10^{-2}]	Validation loss [10^{-2}]
Allocator	4.14	4.08
Length 5	5.39	5.20
Length 10	3.79	3.64
Length 15	3.33	3.23

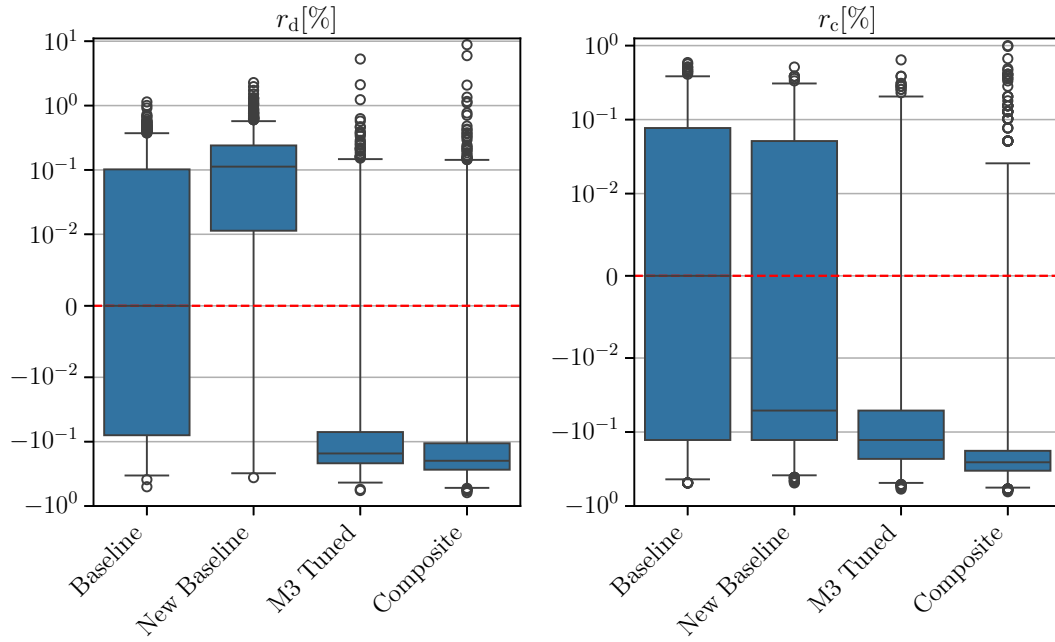


Figure 4.7: Notion of regret for duration and cost for the composite model for multiple problem instances

Table 4.11: Benchmark results for the composite model for multiple problem instances

Model	Success [%]	Duration [ms]	Cost [s]
Baseline	100	136.68	19.50
New Baseline	100	152.05	18.50
Composite	99	109.60	14.50
Model 3 tuned	100	115.83	17.00

5 Discussion

In this chapter, the results from Cha. 4 are discussed and evaluated.

5.1 Models for Specific Problem Instances

The dynamics model is the only model trying to output the states $\mathbf{q}$ and actions $\mathbf{u}$, which are linked by the dynamics f . The error defined in Eq. 3.10 resulting from this link is, therefore, only present in this model. When sampling from the model, this error is ≈ 0.04 for both problem instances. To put some meaning to this value, the maximum possible change from $\mathbf{q}_i = \begin{bmatrix} x_i & y_i & \theta_i \end{bmatrix}^\top$ to q_1 , is achieved at the bounds of $s \in \{-0.5, 0.5\}$ and $\varphi \in \{-0.5, 0.5\}$.

$$s = 0.5, \quad \varphi = 0.5, \quad dt = 0.1 \quad (5.1)$$

$$|\mathbf{q}_1 - \mathbf{q}_2|_2^2 = \left\| \begin{bmatrix} x_i \\ y_i \\ \theta_i \end{bmatrix} - \begin{bmatrix} x_i + s \cos(\theta)dt \\ y_i + s \sin(\theta)dt \\ \theta_i + \varphi dt \end{bmatrix} \right\|_2^2 = \left\| \begin{bmatrix} 0.05 \\ 0 \\ 0.05 \end{bmatrix} \right\|_2^2 = 0.005 \quad (5.2)$$

The error achieved by the model is, therefore, around eight times larger than the maximum distance between two states. The sampled states and actions cannot be used in conjunction. For this reason, the benchmark using db-A* only uses the actions alongside the calculated states.

The minimum loss achieved for the dynamics model is the lowest achieved over all models. This, however, does not correlate at all with its performance in the benchmark, where it performs the worst. While it is able to find solutions for the bugtrap problem, the duration and cost are significantly higher than the baseline. The model trained for the alcove problem is not able to find a solution at all. A few reasons can explain this. The states the model tries to replicate are most certainly too noisy, as the translation invariance is not utilized. The same motion primitive produces different states based on

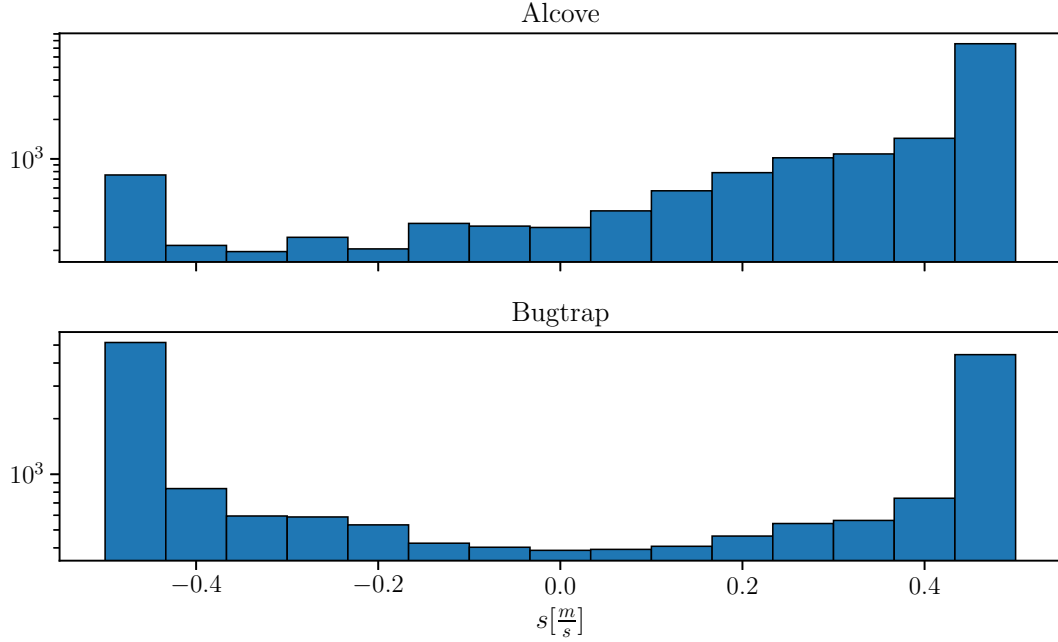
its initial position, which is an unnecessary complication to the model. In addition, the model is not complex enough to accurately model the distribution of the actions given by the dataset in conjunction with the correct correlated states. These are fixable issues, but the inclusion of the states is mostly irrelevant to the task at hand, and the model is therefore unsuitable, as discussed in Sec. 3.3.2.

The losses shown in Fig. 4.1 show an image that is present in all models for the fixed environment. After a maximum of around 1000 epochs, both training and validation loss have reached a plateau, with only slight improvements in further epochs. Both losses are around the same value, which indicates that no overfitting is taking place, and the model is able to generalize quite well to the distribution present in the dataset. The reported minimum validation loss is, in almost all cases, minimally lower than the minimum training loss, which can be explained by the heavier fluctuation of the validation loss.

The losses for models for the alcove problem are lower in all cases than their bugtrap counterparts. This can be explained through the complexity of the underlying dataset. The alcove problem requires mostly motion primitives to steer to the right, while the bugtrap problem needs a higher variety. From a simpler dataset follows an easier-to-learn distribution. The distribution of the mean of s is shown in Fig. 5.1. The alcove problem has a significant bias towards motion primitives with a positive speed, while the distribution found in the bugtrap problem is almost symmetrical.

The validation loss also correlates with the performance in the benchmark if one excludes the model utilizing the states and the one outlier with a success rate of 60% for P_{rel} , as can be seen in Tab. 4.1 and Tab. 4.2. This is promising for the model selection, as benchmarking multiple models is time-consuming and not easily done while training. This is limited, however, for models with a similar architecture, especially the dimension of the output. The losses for the dynamics model utilizing $\mathbf{q}$ do not fit the correlation established by the other models as a consequence of the different architecture. The validation loss can, therefore, be used to select the most suitable model from a range of models where the difference lies only in the chosen hyperparameters, not in the basic architecture. This should also be valid for the conditioning, as it does not affect the dimensionality of the model output.

The basic model, presented in Sec. 3.3.1, shows improvements in duration and cost for the alcove problem but is not able to reproduce them for the bugtrap problem. This can largely be explained by two factors: i) the problem complexity and ii) the absence of conditioning in the model. As the model can only sample from the whole distribution without differentiating between the motion primitives, it is very likely to sample motion

Figure 5.1: Distribution of s in the alcove and bugtrap training data

primitives that lead to a solution but not necessarily a good one. The probability of sampling good motion primitives is way more likely for the simpler problem. This basic model is thus only useful for simplistic problem instances.

The aforementioned outlier in the benchmarks is caused by the model using P_{rel} as conditioning. In stark contrast to all other suitable models and the baseline, it only solves 60% of the alcove problem’s 20 trials. However, the equivalent model for the bugtrap with the same hyperparameters is capable of solving all trials for that specific problem instance. This behavior was reproducible with multiple retrained models over multiple trial runs. This leads to the conclusion that the issue originates in the problem instance or, more specifically, in the dataset for the problem instance. Additionally, it has to originate from the parameter P_{rel} , as no other model had problems solving the alcove problem. The issue is apparent when inspecting the distribution of P_{rel} for both datasets in Fig. 5.2 left and right respectively. There are significantly more motion primitives with a low P_{rel} in both datasets. The bugtrap problem has some data points over almost the whole spectrum, while the alcove problem has large gaps for $P_{\text{rel}} > 0.4$. The possible consequences of gaps in the distribution, specifically for P_{rel} , were discussed in Sec. 3.4.1, and the resulting hallucinations are the most probable cause for the low success rate.

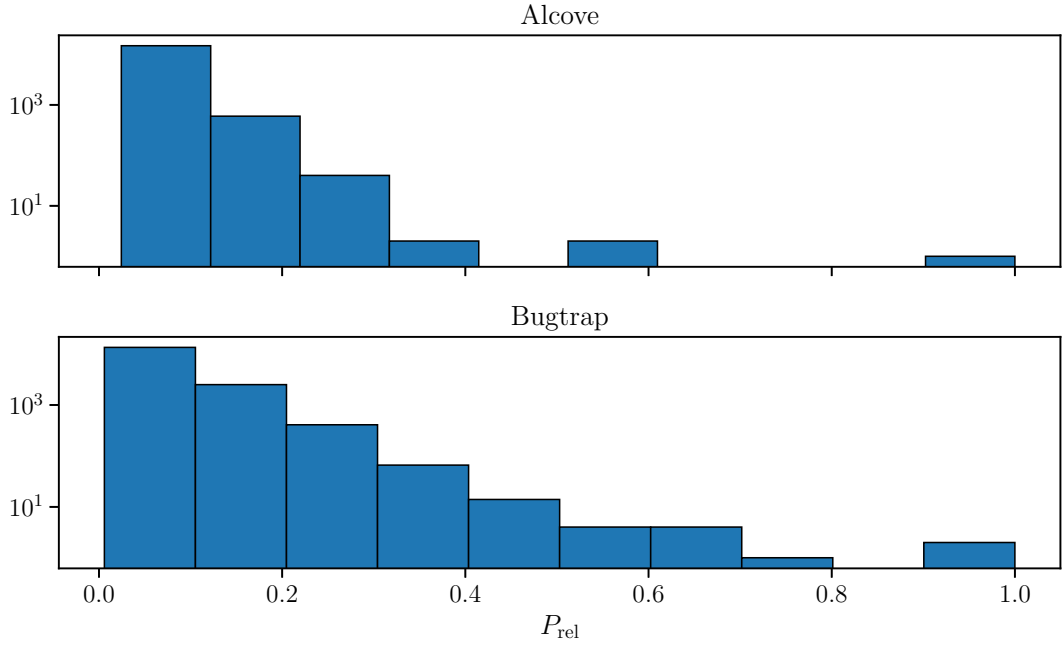


Figure 5.2: Distribution of P_{rel} in the alcove and bugtrap training data

The low duration achieved by the model is most likely caused by the sampled dataset having just the right amount of viable motion primitives to find a solution. With the rest of the dataset consisting of motion primitives that cannot be applied, a solution can be found very quickly. Due to its low reliability, P_{rel} is not present in other models for the fixed environment.

The model presented in Sec. 3.4.2 proves to be more stable, as a uniform distribution of L_{rel} is guaranteed. This model is able to solve both problem instances consistently and at a lower cost than the baseline. The worst result for the alcove problem is as good as the median result of the baseline, while the results in the Interquartile Range (IQR) show an improvement of around 25%. The duration is even clearer in its improvement, with the worst outlier still showing an improvement of 25% and the bulk of the trials achieving runtimes 40% faster than the baseline. Once again, the bugtrap problem shows fewer improvements. The median of the cost improves upon the baseline by around 10%, although there are some trials with a higher cost than the median baseline, up to around 4% higher. However, the IQR still resides completely in the negative. The duration for the bugtrap problem, however, does not improve upon the baseline in any way. While the median is higher than the median of the baseline, the difference is very minimal.

Broadly speaking, the model finds a solution as quickly as the baseline. This model is most promising, as its performance is equal or greater to the baseline in all metrics without tuning. The last two models presented in this section also utilize L_{rel} for this reason.

The model from Sec. 3.4.3 tries to utilize varying discontinuity bounds δ . This, however, does not seem to help improve the results. While the model shows similar results for the alcove problem as the previous model utilizing only L_{rel} , the bugtrap problem does not. The median for both metrics still shows improvements, but the IQR is as widely spread as IQR of the baseline itself. The probable main cause of this result is the generation of the dataset. Alg. 3 adds optimized motion primitives after each iteration to the dataset. This, however, was not done in this case, as it was intended to improve upon the original dataset. The addition of the optimization would most likely be very beneficial to the quality and, therefore, performance of all models, as the optimized motion primitives should provide a clearer distribution. This was recognized but not executed due to time limitations.

The last model presented for a fixed environment from Sec. 3.4.4 shows the best results overall. All median results of this model are the best over all models, which can be seen in Tab. 4.2, indicated in bold text. Over both problem instances, the IQR of all metrics is completely in the negative. All outliers for the alcove problem are negative as well. Just a small number of trials in the bugtrap problem show worse performance than the baseline. The most apparent improvement is visible in the duration for the bugtrap problem, shown in Fig. 4.3. The cost for the alcove problem shows a really consistent result, as the model achieves the same cost for all trials except one. This cost is also consistent with the best cost achieved by the baseline over all trials. The replacement of θ_0 with Θ_0 is a clear general improvement and will therefore be used in all further models. The cause for the general improvement is apparent, as it allows the model to learn better starting angles close to the bounds of $-\pi$ and π .

In conclusion, the models intended for a fixed problem show that the general concept works as intended. All models in this section are used with the same arbitrarily set hyperparameters and are not fine-tuned. This was not done, as the fixed problem was just used as a proof of concept. The main goal of this thesis is to generate motion primitives on arbitrary problem instances, and therefore, fine-tuning of the hyperparameters will be explored in the next section.

5.2 Models for Multiple Problem Instances

The models from the last section showed that learning $\mathbf{u}$, Θ_0 and L_{rel} are a solid foundation. A large number of hyperparameters are present to tackle the actual task of generating datasets for multiple problem instances. The chosen method of using Optuna together with ASHA to explore parameter combinations indicates that only two parameters that describe the environment have a significant effect on the model quality. The first, $P(\mathcal{W}_{\text{blocked}})$, was expected to be a good parameter, as it is the only one containing information about the obstacles, which play an important role in the possible solution paths. θ_0^{goal} has a higher impact than θ_0^{start} , which was not expected. This suggests that only the goal angle has a significant correlation with the motion primitive distribution. There is currently no obvious explanation for this behavior. The reason could lie in the generation process of the random environments, where there might be a bias towards solution paths that tend towards the goal angle. This is, however, just a guess that has not been explored further.

Additionally, P_{rel} and L_{rel} are also inspected as part of the selectable parameters. As the dataset is more diverse and split up due to conditioning and multiple instances, the relative probability P_{rel} may be more stable than for the fixed environment. This proves to be true as it appears quite a few times in the top models, although always in conjunction with L_{rel} . All four models selected once again show the same behavior in their losses as already observed in the models for the fixed environment. Both training and validation loss are relatively small spread over all models, with around 10% difference.

This small spread leads to no visible correlation between validation loss and performance, unlike before. Still, the model with the lowest validation loss, model 3, shows the best benchmark results. This leads to the conclusion that the validation loss is a good first indicator of a model’s performance but by no means a perfect one. The actual performance can only be observed in the actual benchmark.

Model 3, which utilizes L_{rel} , $P(\mathcal{W}_{\text{blocked}})$ and θ_0^{goal} performed the best in the benchmark. Its duration is around 10% faster than the baseline, with its IQR including some trials slower than the baseline median. Its cost achieves a 12.5% improvement, with only a few trials being more costly than the baseline median. The entire IQR is in the negative. All models except model 1 can be described in this way, although model 3 has a small advantage over all of them. For this reason, model 3 is selected for further fine-tuning.

The fine-tuning through the same process as before changes the size and dimension of

the hidden layers, as well as the learning rate, as noted in Tab. 4.6. The cost, visible in Fig. 4.6 (right), shows almost no improvement. The only visible change is a slight reduction in outliers. The duration, however, shows significant improvements, which can be seen in Fig. 4.6 (left). While the median improves upon the 10% in the untuned model to around 15% in the tuned model, the IQR almost surpasses the 10% mark. Still, a good amount of trials are taking longer than the baseline median.

The final model inspected in this thesis, presented in Sec. 3.4.6, is fine-tuned from the start, as it can be done independently from the benchmark. The sub-model allocating the sample sizes of the further models is a very small model with 2 hidden layers of size 16. When inspecting the dataset, there seems to be a consistent composition of the three lengths in every solution, which does not vary substantially. Therefore, a simple MLP could possibly achieve the same performance as the used diffusion model. This is not explored, as the previous models could be used for a swift implementation. When fine-tuning the models that actually generate motion primitives, there seems to be a trend in their required architecture, as shown in Tab. 4.9. The length 5 model uses the hyperparameters specified in the tuned model 3. The models with lengths 10 and 15 then profit increasingly from a more complex hidden layer structure. An inverse effect is observable in the learning rate. With increasing length, the minimum achieved loss shown in Tab. 4.10 also improves by a not insignificant amount compared to the length 5 model. This can be explained by the correlation between the single action that makes up a motion primitive. It is highly probable that an action u_{i+1} will be similar to its previous action u_i . This stems from the fact that the motion primitives used to train the models are optimized for cost, which leads to smooth transitions. Therefore, the longer a motion primitive, the more the diffusion model can profit from the correlation, and it is able to achieve lower losses.

The baseline used in the previous models is still used as the main comparison, while the new baseline, which includes different lengths for the motion primitives, is also shown in Tab. 4.11 and Fig. 4.7. Additionally, the tuned model 3 is used as a comparison as well. The multiple-length baseline is able to achieve lower cost solutions compared to the single-length. It, however, trades this advantage for a longer runtime. In total, the improvement in cost is not as significant as the loss in duration. The underlying reason for the increase in computation time is not completely clear at the time, as higher length motion primitives intuitively should lead to quicker solutions since fewer vertices and, thus, fewer exploration steps are needed.

The composite model is able to circumvent the issues shown by the multiple-length

baseline largely. In fact, it shows the best performance observed in this thesis. For both cost and duration, the IQR is entirely outside the IQR of both baselines. It achieves costs around 25% lower and a duration of around 20% lower than the original baseline. Both are an improvement from the previously tuned model 3. Nonetheless, some trials are performing worse than the baseline median in both metrics. The outliers even include some trials that perform worse than any trial from both baselines, although they make up less than 1% of the trials.

6 Conclusion and Future Work

This thesis presents a novel approach to sampling motion primitives for kinodynamic motion planning through the usage of DDPM. In the process, multiple iterations of such a model are discussed and tested. The models are integrated into the existing graph-based planning algorithm db-A* to evaluate the quality of the generated motion primitives. The main idea of this work is to learn the underlying distribution of actions $\mathbf{u}$ and its starting angle θ_0 of the motion primitives that are used to reach the goal in a specific problem instance. This is sufficient, as db-A* expect translation invariant motion primitives. This distribution is acquired by querying db-A* with a standard motion primitive dataset.

The aspired goal of this thesis is to be able to generate a motion primitive dataset for an arbitrary problem instance, which then enables db-A* to find solutions with a lower cost and in a shorter time than the standard dataset.

As a proof of concept, a handful of models are trained on a specific problem instance. In this process, a few key parameters are explored to differentiate between the motion primitives. The parameter L_{rel} , which differentiates a motion primitive based on its relative location in a given solution, stands out as it allows for solid improvements while maintaining stability. An emerging issue related to the circular nature of θ_0 can be largely resolved by utilizing the alternative representation $\mathbb{R}^2(\theta_0)$. The shown untuned models are largely able to improve upon the performance set by the standard dataset.

These promising results then lead to the creation of models that support arbitrary problem instances. A basic algorithm to generate random problem instances is created and used to generate the required dataset. Problem-describing parameters are selected and explored by training models with the hyperparameter exploring search algorithm Optuna and the training scheduler ASHA. The best model is then tuned by optimizing its hidden layers and learning rate. The resulting model shows approximately 10% improvement in both duration and cost over the original dataset in solving the randomly generated problem instances.

As the last improvement to this work, multiple models are combined. Three models are trained and optimized to generate a specific length motion primitive. A fourth model

is trained to allocate each sample to one of the three models. This allows for even higher improvements in both metrics, with a cost decrease of 25% and a duration decrease of 20% over the baseline.

In conclusion, the benchmarks show that diffusion models can be used to sample motion primitives, which significantly improve upon the performance of kinodynamic motion planners. With relatively basic parameters describing an arbitrary motion planning problem, both cost and planning time can be reduced consistently to around 80% of the results achieved by conventional methods. Achieving these kinds of improvements with basic diffusion model methods is a compelling demonstration to further explore this approach and expand upon this work.

Future Work

The process of creating these models and the promising outlook of this approach opens up various directions for improving the presented models. A few of these ideas will be discussed in this section.

Test Additional Dynamics

All results shown in this thesis are based on the dynamics of a first-order unicycle, a relatively simple case. Even though db-A* only utilizes the translation invariance of the dynamics, the first-order unicycle is also rotation invariant, further simplifying the resulting motion primitive distribution. To validate the approach proposed by this work, additional, more complex dynamics have to be tested. A second-order unicycle is a logical next step to expand on. Other possible systems to inspect include, for example, a quadrotor [60] or a car with a trailer [61].

Incorporation into kMP-db-A*

In [5], the kinodynamic motion planning algorithm db-A* is used inside the anytime motion planning algorithm kMP-db-A*. The discontinuity bound δ is systematically decreased while optimized motion primitives from previous iterations are added to the dataset. The current diffusion model does not take this into account, as it only utilizes db-A*. It can be clearly seen in both the tuned model 3 as well as the composite model, that the current approach is able to consistently replicate the best solutions provided by the standard dataset. Solutions provided by the optimizer present in kMP-db-A*

are of lower cost than the ones used in the current dataset. Additionally, they should provide a clearer distribution as no discontinuities are present after the optimization process. Including the optimized motion primitives in the training data could, therefore, be beneficial.

Expanded Problem Instance Parametrization

The parameters selected for problem instance parametrization are of a relatively basic nature. By employing more complex approaches to the parametrization for conditioning, the samples provided by the model are allowed to be even more specific to the problem instance. The current parametrization is, for example, unable to depict if the free workspace $\mathcal{W}_{\text{free}}$ is in the shape of a long, tight corridor or a single large room. This, however, could be valuable to the motion primitive distribution. One could extract this information from the creation process of the random problem instances, although this would disallow the use of real arbitrary problems. The goal should be to gather this information from already created problem instances that only contain the information required to create it, for example, as defined in Eq. 2.9. The primary objective is to find a way to find a compact representation of the complete problem definition and use it as conditioning. A possible approach to this could be using convolutional layers [62]. An image of the problem instance can be used as conditional input to the diffusion model, potentially leading to more detailed information about the problem instance. A similar approach is using an autoencoder [63], where the latent space of the same image representation is used as conditioning to the diffusion model. Problem instances can be of arbitrary size, leading to complications in these approaches. The dimensionality of the models is traditionally fixed, but this limitation can be somewhat alleviated by employing a pooling layer or scaling the images.

Multi-Robot Case

The motion planning algorithm db-A* is used in the multi-robot planning algorithm db-CBS. The current approach only considers the problem instance for the single robot case and, as such, is not necessarily suitable for the multi-robot case. To generate motion primitives that are well suited for a problem definition that includes multiple robots, the conditioning has to be expanded. The objective is, similar to Sec. 6, to find a compact representation for the other robots and their respective tasks, that are now present in the environment. This representation has to be chosen carefully. If not, this may result

in a model that is limited to a specific number of robots in the problem definition.

Bibliography

- [1] B. Donald, P. Xavier, J. Canny, and J. Reif, “Kinodynamic motion planning,” *J. ACM*, vol. 40, no. 5, pp. 1048–1066, Nov. 1993, ISSN: 0004-5411.
- [2] P. E. Hart, N. J. Nilsson, and B. Raphael, “A Formal Basis for the Heuristic Determination of Minimum Cost Paths,” *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, Jul. 1968, ISSN: 2168-2887.
- [3] S. M. LaValle, “Rapidly-exploring random trees : A new tool for path planning,” *The annual research report*, 1998.
- [4] M. Toussaint, *Newton methods for k-order Markov Constrained Motion Problems*, Jul. 2014. arXiv: 1407.0414.
- [5] W. Hönig, J. Ortiz-Haro, and M. Toussaint, “Db-A*: Discontinuity-bounded Search for Kinodynamic Mobile Robot Motion Planning,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Kyoto, Japan: IEEE, Oct. 2022, pp. 13 540–13 547, ISBN: 978-1-66547-927-1.
- [6] A. Moldagalieva, J. Ortiz-Haro, M. Toussaint, and W. Hönig, “Db-CBS: Discontinuity-bounded conflict-based search for multi-robot kinodynamic motion planning,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 14 569–14 575.
- [7] Z. C. Goddard, K. Wardlaw, K. Williams, and A. Mazumdar, “Selecting Minimal Motion Primitive Libraries with Genetic Algorithms,” *Journal of Aerospace Information Systems*, vol. 20, no. 10, pp. 618–626, Oct. 2023, ISSN: 1940-3151, 2327-3097.
- [8] M. Poffald, Y. Zhang, and K. Hauser, “Learning problem space metrics for motion primitive selection,” in *IROS Workshop on Machine Learning in Planning and Control of Robot Motion*, vol. 2, 2014, p. 3.
- [9] J. Ho, A. Jain, and P. Abbeel, “Denoising diffusion probabilistic models,” *Advances in neural information processing systems*, vol. 33, pp. 6840–6851, 2020.

- [10] S. M. LaValle, *Planning Algorithms*, 1st ed. Cambridge University Press, May 2006, ISBN: 978-0-521-86205-9.
- [11] K. M. Lynch and F. C. Park, *Modern Robotics: Mechanics, Planning, and Control*, 1st ed. Cambridge University Press, May 2017, ISBN: 978-1-316-66123-9.
- [12] R. Dechter and J. Pearl, “Generalized best-first search strategies and the optimality of A*,” *Journal of the ACM*, vol. 32, no. 3, pp. 505–536, Jul. 1985, ISSN: 0004-5411, 1557-735X.
- [13] A. Martelli, “On the complexity of admissible search algorithms,” *Artificial Intelligence*, vol. 8, no. 1, pp. 1–13, Feb. 1977, ISSN: 00043702.
- [14] L. Kavraki, P. Svestka, J.-C. Latombe, and M. Overmars, “Probabilistic roadmaps for path planning in high-dimensional configuration spaces,” *IEEE Transactions on Robotics and Automation*, vol. 12, no. 4, pp. 566–580, Aug. 1996, ISSN: 2374-958X.
- [15] J. D. Gammell and M. P. Strub, “Asymptotically optimal sampling-based motion planning methods,” *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 4, no. 1, pp. 295–318, 2021.
- [16] S. Karaman and E. Frazzoli, “Sampling-based algorithms for optimal motion planning,” *The international journal of robotics research*, vol. 30, no. 7, pp. 846–894, 2011.
- [17] K. Solovey, L. Janson, E. Schmerling, E. Frazzoli, and M. Pavone, “Revisiting the asymptotic optimality of RRT,” in *IEEE International Conference on Robotics and Automation (ICRA)*, IEEE, 2020, pp. 2189–2195.
- [18] N. Ratliff, M. Zucker, J. A. Bagnell, and S. Srinivasa, “CHOMP: Gradient optimization techniques for efficient motion planning,” in *IEEE International Conference on Robotics and Automation*, Kobe: IEEE, May 2009, pp. 489–494, ISBN: 978-1-4244-2788-8.
- [19] J. Polden, Z. Pan, N. Larkin, and S. van Duin, “Adaptive Partial Shortcuts: Path Optimization for Industrial Robotics,” *Journal of Intelligent & Robotic Systems*, vol. 86, no. 1, pp. 35–47, Apr. 2017, ISSN: 1573-0409.
- [20] A. T. Le, G. Chalvatzaki, A. Biess, and J. R. Peters, “Accelerating motion planning via optimal transport,” *Advances in Neural Information Processing Systems*, vol. 36, 2024.

- [21] A. Li, Z. Ding, A. B. Dieng, and R. Beeson, *DiffuSolve: Diffusion-based Solver for Non-convex Trajectory Optimization*, Oct. 2024. arXiv: 2403.05571.
- [22] W. Hönig and A. Orthey, *Motion Planning (Lecture) - Organization, Introduction, Problem Formulation*, Apr. 2023.
- [23] B. M. Kim and P. Tsiotras, “Controllers for unicycle-type wheeled robots: Theoretical results and experimental validation,” *IEEE Transactions on Robotics and Automation*, vol. 18, no. 3, pp. 294–307, Jun. 2002, ISSN: 2374-958X.
- [24] S. Kucuk and Z. Bingul, *Robot Kinematics: Forward and Inverse Kinematics*. IN-TECH Open Access Publisher London, UK, 2006.
- [25] M. Pivtoraiko and A. Kelly, “Kinodynamic motion planning with state lattice motion primitives,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems*, Sep. 2011, pp. 2172–2179.
- [26] B. J. Cohen, G. Subramania, S. Chitta, and M. Likhachev, “Planning for Manipulation with Adaptive Motion Primitives,” in *IEEE International Conference on Robotics and Automation*, May 2011, pp. 5478–5485.
- [27] B. J. Cohen, S. Chitta, and M. Likhachev, “Search-based planning for manipulation with motion primitives,” in *IEEE International Conference on Robotics and Automation*, May 2010, pp. 2902–2908.
- [28] D. J. Grymin, C. B. Neas, and M. Farhood, “A hierarchical approach for primitive-based motion planning and control of autonomous vehicles,” *Robotics and Autonomous Systems*, vol. 62, no. 2, pp. 214–228, Feb. 2014, ISSN: 09218890.
- [29] S. Karaman, M. R. Walter, A. Perez, E. Frazzoli, and S. Teller, “Anytime Motion Planning using the RRT*,” in *IEEE International Conference on Robotics and Automation*, May 2011, pp. 1478–1483.
- [30] K. Hornik, M. Stinchcombe, and H. White, “Multilayer feedforward networks are universal approximators,” *Neural Networks*, vol. 2, no. 5, pp. 359–366, Jan. 1989, ISSN: 08936080.
- [31] Y. Lu and J. Lu, “A universal approximation theorem of deep neural networks for expressing probability distributions,” *Advances in neural information processing systems*, vol. 33, pp. 3094–3105, 2020.

- [32] R. Kruse, S. Mostaghim, C. Borgelt, C. Braune, and M. Steinbrecher, “Multi-layer perceptrons,” in *Computational Intelligence: A Methodological Introduction*, Cham: Springer International Publishing, 2022, pp. 53–124, ISBN: 978-3-030-42227-1.
- [33] Hecht-Nielsen, “Theory of the backpropagation neural network,” in *International 1989 Joint Conference on Neural Networks*, Jun. 1989, 593–605 vol.1.
- [34] Q. Wang, Y. Ma, K. Zhao, and Y. Tian, “A Comprehensive Survey of Loss Functions in Machine Learning,” *Annals of Data Science*, vol. 9, no. 2, pp. 187–212, Apr. 2022, ISSN: 2198-5812.
- [35] C. M. Bishop and H. Bishop, *Deep Learning: Foundations and Concepts*. Cham: Springer International Publishing, 2024, ISBN: 978-3-031-45467-7.
- [36] D. P. Kingma and J. Ba, *Adam: A Method for Stochastic Optimization*, Jan. 2017. arXiv: 1412.6980.
- [37] X. Ying, “An Overview of Overfitting and its Solutions,” *Journal of Physics: Conference Series*, vol. 1168, no. 2, p. 022 022, Feb. 2019, ISSN: 1742-6596.
- [38] Z. Tu, “Learning Generative Models via Discriminative Approaches,” in *IEEE Conference on Computer Vision and Pattern Recognition*, Jun. 2007, pp. 1–8.
- [39] J. Sohl-Dickstein, E. Weiss, N. Maheswaranathan, and S. Ganguli, “Deep unsupervised learning using nonequilibrium thermodynamics,” in *International Conference on Machine Learning*, PMLR, 2015, pp. 2256–2265.
- [40] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, “High-resolution image synthesis with latent diffusion models,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 10 684–10 695.
- [41] A. Ramesh, P. Dhariwal, A. Nichol, C. Chu, and M. Chen, *Hierarchical Text-Conditional Image Generation with CLIP Latents*, Apr. 2022. arXiv: 2204.06125.
- [42] J. Austin, D. D. Johnson, J. Ho, D. Tarlow, and R. van den Berg, “Structured Denoising Diffusion Models in Discrete State-Spaces,” in *Advances in Neural Information Processing Systems*, vol. 34, Curran Associates, Inc., 2021, pp. 17 981–17 993.
- [43] Z. Kong, W. Ping, J. Huang, K. Zhao, and B. Catanzaro, “DiffWave: A versatile diffusion model for audio synthesis,” in *International Conference on Learning Representations*, 2021.

- [44] S. P. Meyn and R. L. Tweedie, *Markov Chains and Stochastic Stability* (Communications and Control Engineering Series), 2nd ed. Cambridge ; New York: Cambridge University Press, 2009, ISBN: 978-0-521-73182-9.
- [45] T. Chen, *On the Importance of Noise Scheduling for Diffusion Models*, May 2023. arXiv: 2301.10972.
- [46] Z. Zhang, Z. Zhao, J. Yu, and Q. Tian, “ShiftDDPMs: Exploring conditional diffusion models by shifting diffusion trajectories,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, 2023, pp. 3552–3560.
- [47] S. Dieleman, *Guidance: A cheat code for diffusion models*, <https://benanne.github.io/2022/05/26/guidance.html>, 2022 (accessed 2024-11-20).
- [48] N. Maleki, B. Padmanabhan, and K. Dutta, “AI hallucinations: A misnomer worth clarifying,” in *IEEE Conference on Artificial Intelligence (CAI)*, IEEE, 2024, pp. 133–138.
- [49] A. De Wynter, X. Wang, A. Sokolov, Q. Gu, and S.-Q. Chen, “An evaluation on large language model outputs: Discourse and memorization,” *Natural Language Processing Journal*, vol. 4, p. 100 024, Sep. 2023, ISSN: 29497191.
- [50] M. V. A. Pedrosa, T. Schneider, and K. Flaßkamp, “Learning Motion Primitives Automata for Autonomous Driving Applications,” *Mathematical and Computational Applications*, vol. 27, no. 4, p. 54, Aug. 2022, ISSN: 2297-8747.
- [51] J. MacQueen, “Some methods for classification and analysis of multivariate observations,” in *Proceedings of 5-Th Berkeley Symposium on Mathematical Statistics and Probability/University of California Press*, 1967.
- [52] E. Camci and E. Kayacan, “Learning motion primitives for planning swift maneuvers of quadrotor,” *Autonomous Robots*, vol. 43, no. 7, pp. 1733–1745, Oct. 2019, ISSN: 1573-7527.
- [53] G. Li, Z. Jin, M. Volpp, F. Otto, R. Lioutikov, and G. Neumann, “Prodmp: A unified perspective on dynamic and probabilistic movement primitives,” *IEEE Robotics and Automation Letters*, vol. 8, no. 4, pp. 2325–2332, 2023.
- [54] E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numerische mathematik*, vol. 1, no. 1, pp. 269–271, 1959.

- [55] J. Ortiz-Haro, W. Hönig, V. N. Hartmann, and M. Toussaint, “iDb-A*: Iterative search and optimization for optimal kinodynamic motion planning,” *IEEE Transactions on Robotics*, 2024.
- [56] A. R. Geist, J. Frey, M. Zobro, A. Levina, and G. Martius, *Learning with 3D rotations, a hitchhiker’s guide to $SO(3)$* , Apr. 2024. arXiv: 2404.11735.
- [57] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, *et al.*, “Pytorch: An imperative style, high-performance deep learning library,” *Advances in neural information processing systems*, vol. 32, 2019.
- [58] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A next-generation hyperparameter optimization framework,” in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, KDD 2019, Anchorage, AK, USA, August 4-8, 2019*, A. Teredesai, V. Kumar, Y. Li, R. Rosales, E. Terzi, and G. Karypis, Eds., ACM, 2019, pp. 2623–2631.
- [59] L. Li, K. Jamieson, A. Rostamizadeh, E. Gonina, J. Ben-tzur, M. Hardt, B. Recht, and A. Talwalkar, “A system for massively parallel hyperparameter tuning,” in *Proceedings of Machine Learning and Systems*, I. Dhillon, D. Papailiopoulos, and V. Sze, Eds., vol. 2, 2020, pp. 230–246.
- [60] P. Wang, Z. Man, Z. Cao, J. Zheng, and Y. Zhao, “Dynamics modelling and linear control of quadcopter,” in *International Conference on Advanced Mechatronic Systems (ICAMechS)*, Nov. 2016, pp. 498–503.
- [61] A. Keymasi Khalaji and S. A. A. Moosavian, “Dynamic modeling and tracking control of a car with n trailers,” *Multibody System Dynamics*, vol. 37, no. 2, pp. 211–225, Jun. 2016, ISSN: 1573-272X.
- [62] K. O’Shea and R. Nash, *An Introduction to Convolutional Neural Networks*, Dec. 2015. arXiv: 1511.08458.
- [63] D. Bank, N. Koenigstein, and R. Giryes, “Autoencoders,” *Machine learning for data science handbook: data mining and knowledge discovery handbook*, pp. 353–374, 2023.

Appendix A Loss Plots

Loss plots

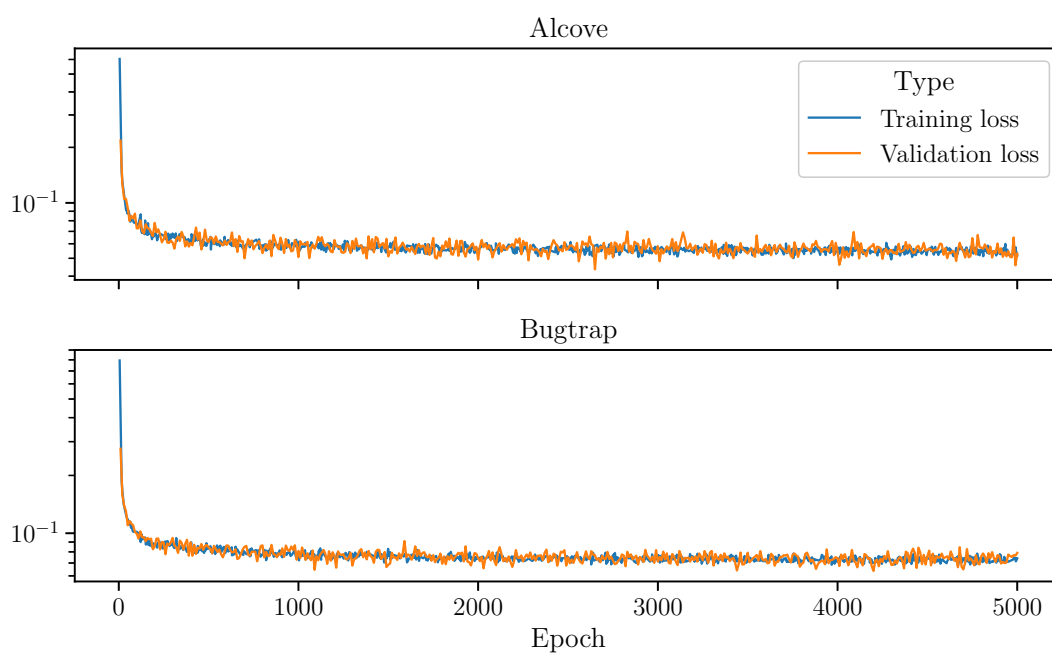
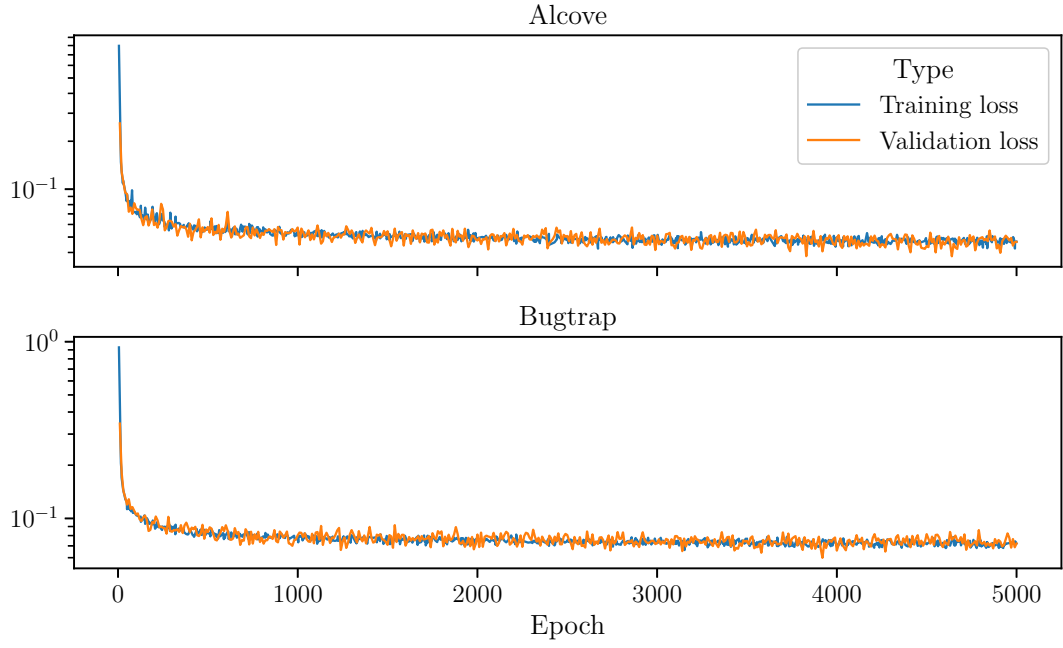
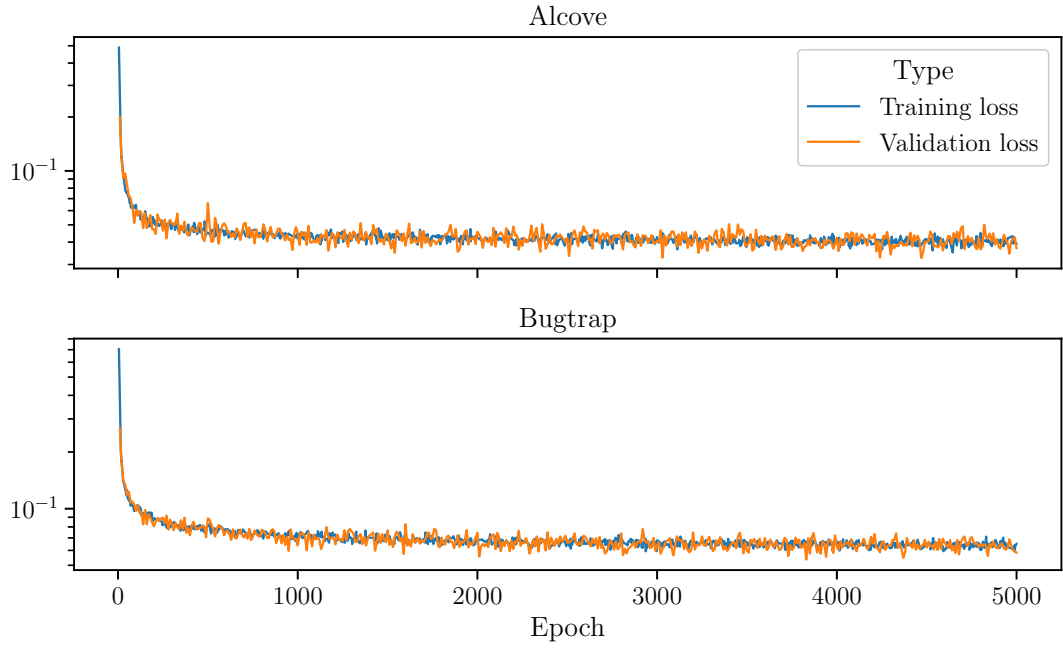


Figure A.1: Training and validation loss of the basic model

Figure A.2: Training and validation loss of the model with P_{rel} Figure A.3: Training and validation loss of the model with L_{rel}

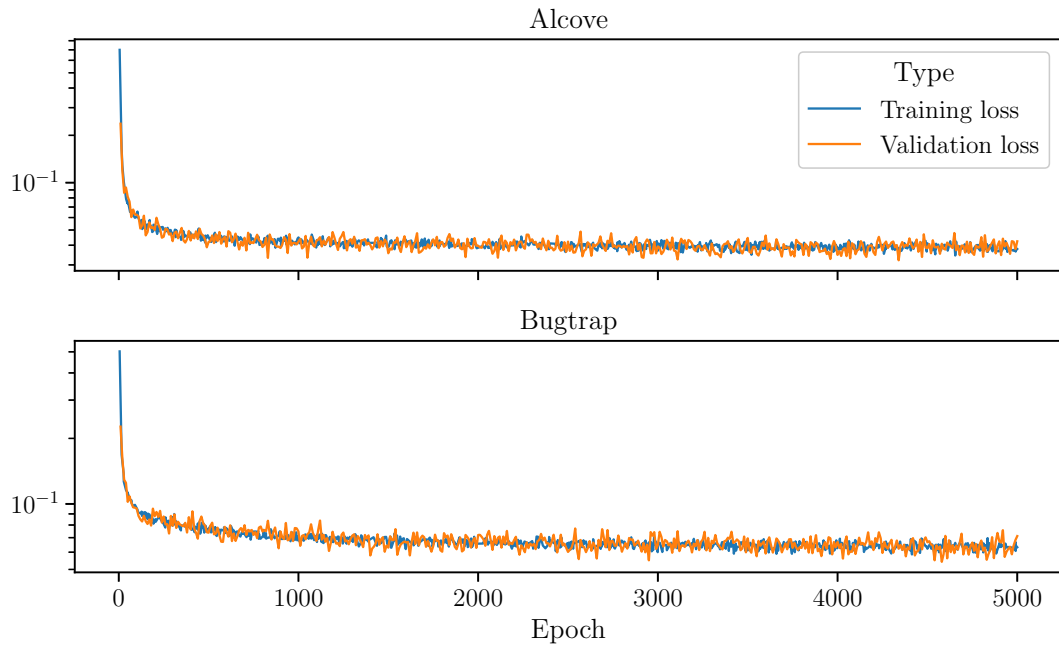


Figure A.4: Training and validation loss of the model with δ

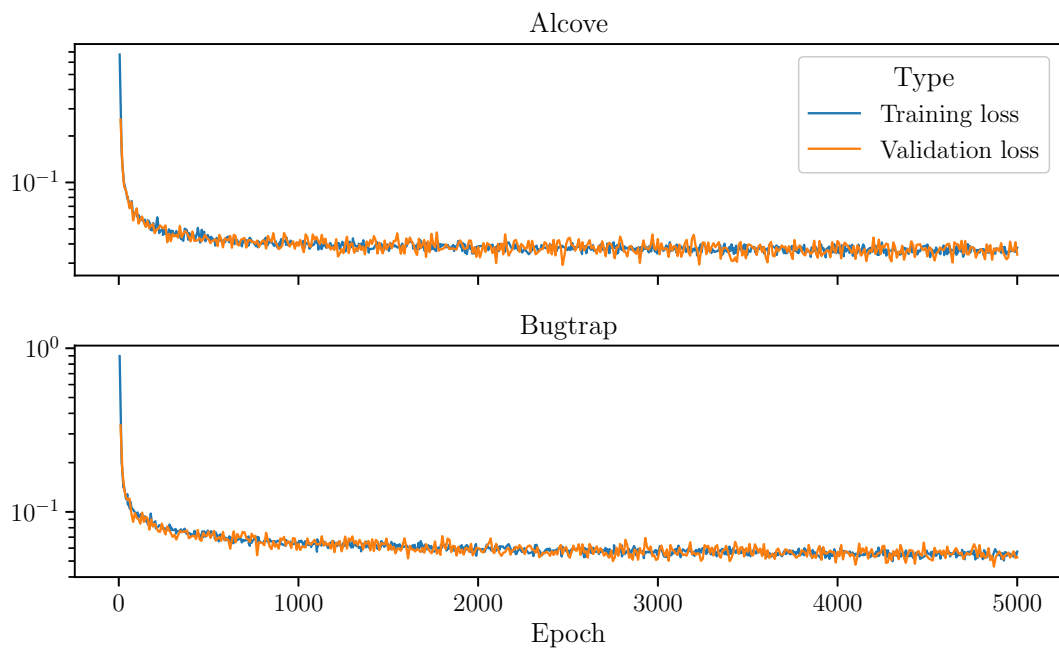


Figure A.5: Training and validation loss of the model with Θ_0

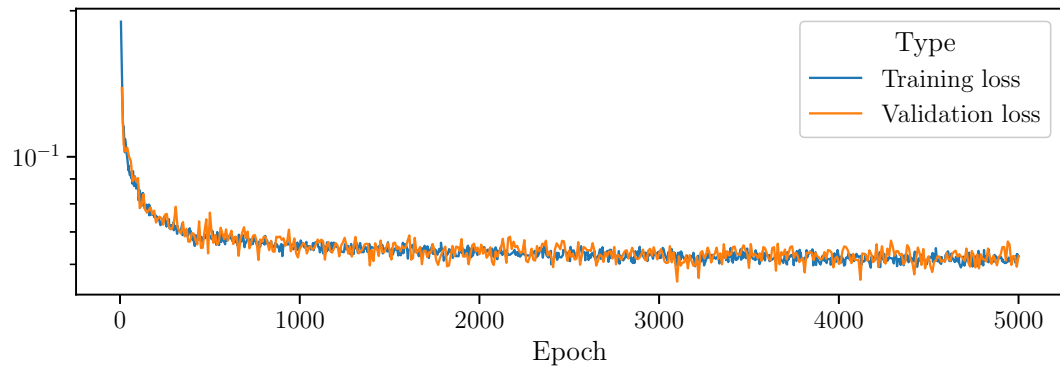


Figure A.6: Training and validation loss of model 1

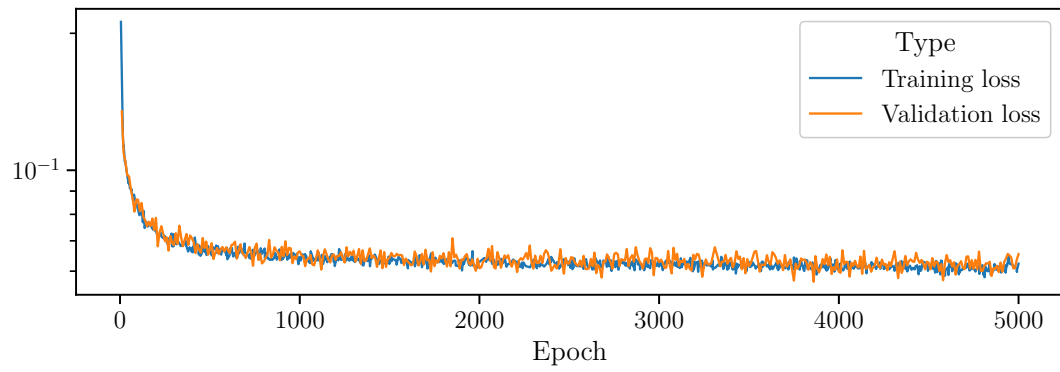


Figure A.7: Training and validation loss of model 2

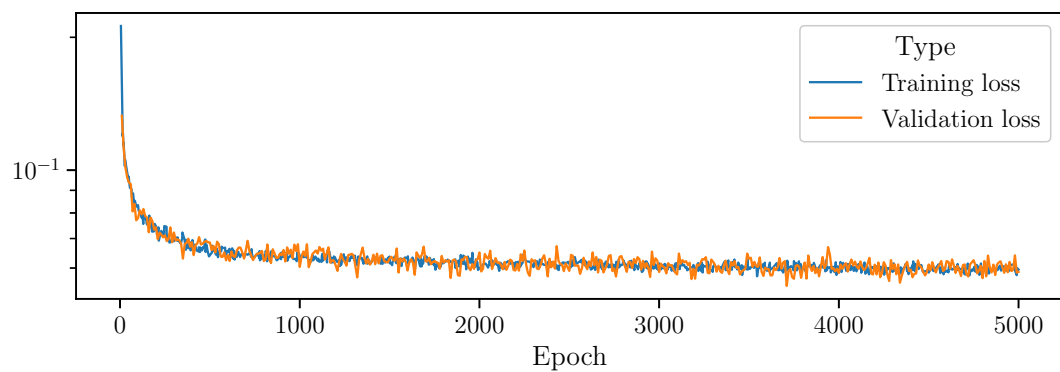


Figure A.8: Training and validation loss of model 3

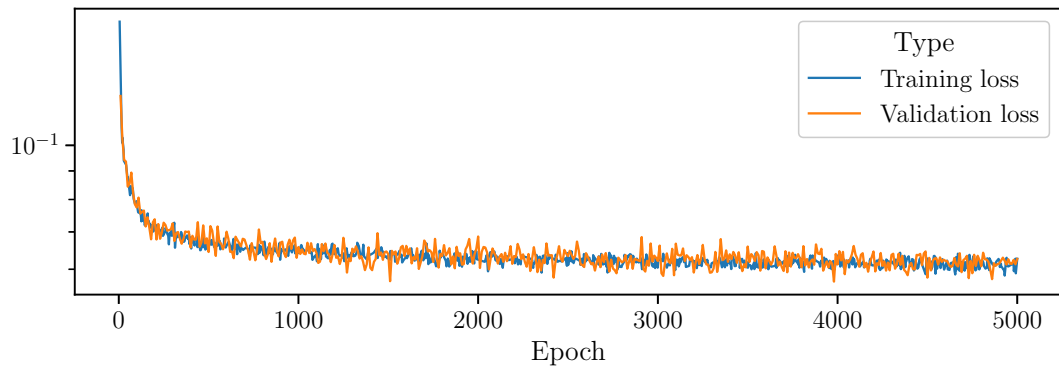


Figure A.9: Training and validation loss of model 4

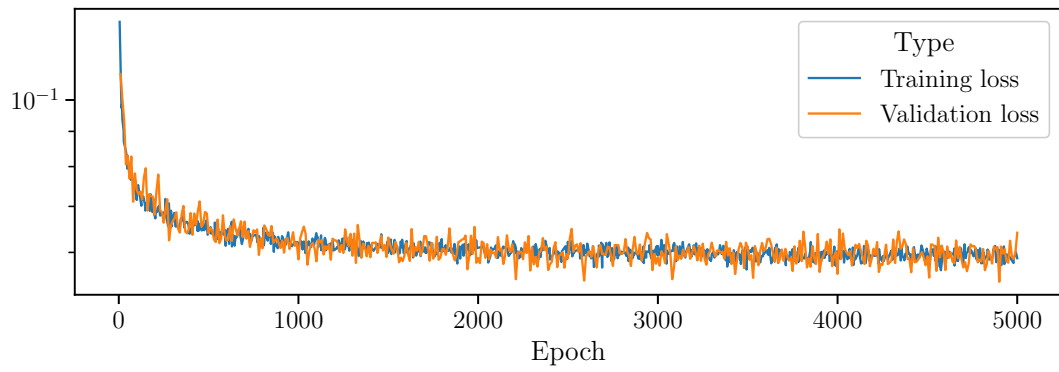


Figure A.10: Training and validation loss of the tuned model 3

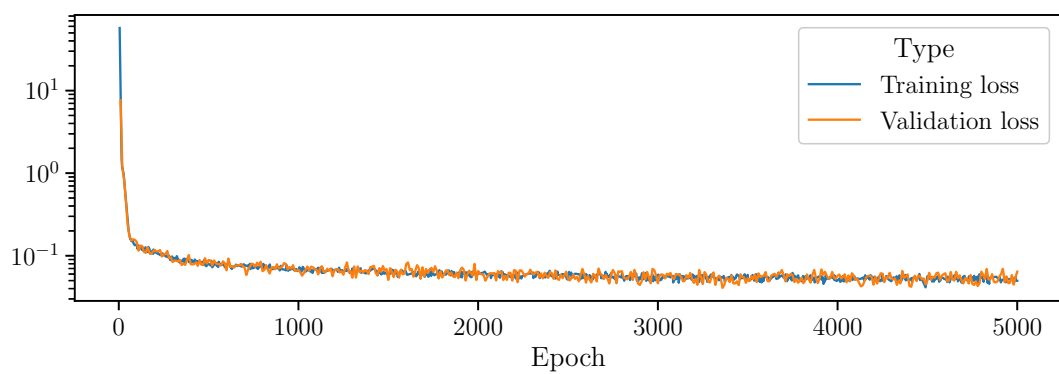


Figure A.11: Training and validation loss of the allocator model

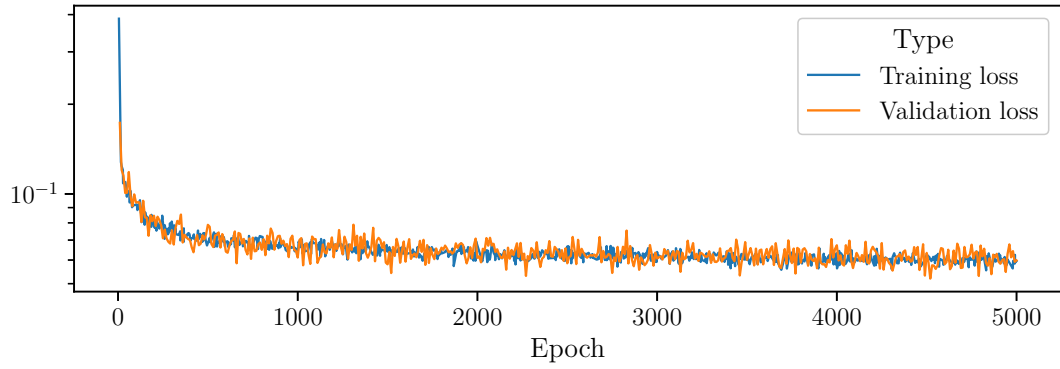


Figure A.12: Training and validation loss of the composite model length 5

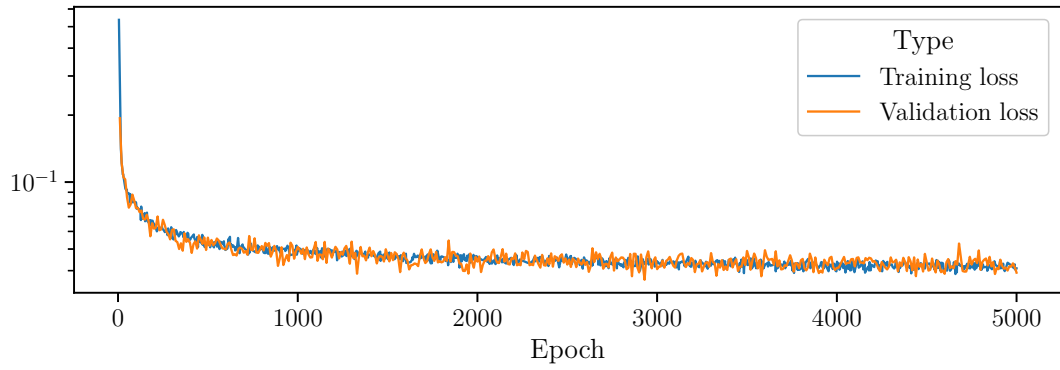


Figure A.13: Training and validation loss of the composite model length 10

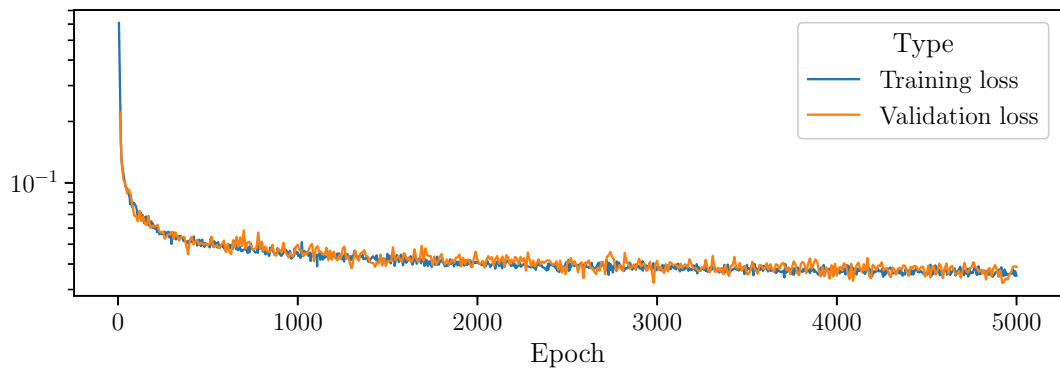


Figure A.14: Training and validation loss of the composite model length 15